

ECOLE NORMALE SUPÉRIEURE DE L'ENSEIGNEMENT
TECHNIQUE DE MOHAMMEDIA
UNIVERSITÉ HASSAN II DE CASABLANCA



المدرسة العليا للأساتذة التعليم التقني
المحمدية
جامعة الحسن الثاني بالدار البيضاء

Département Mathématiques & Informatique

Cours structures de données

Chap 6 : Les arbres

Plan



Introduction

- ✓ Définitions / propriétés
- ✓ Représentation d'un Arbre par un tableau
- ✓ Représentation d'un Arbre par une liste chaînée



Arbres binaires

- ✓ Définitions / propriétés
- ✓ Opérations et Parcours en profondeur/Largeur
- ✓ Complexité



Arbre binaire de recherche

- ✓ Définition / propriétés
- ✓ Opérations sur les ARB
- ✓ Complexité



Arbre binaire de recherche équilibré

- ✓ Définitions / propriétés
- ✓ Les types de rotation et équilibrage



Applications



INTRODUCTION

Introduction

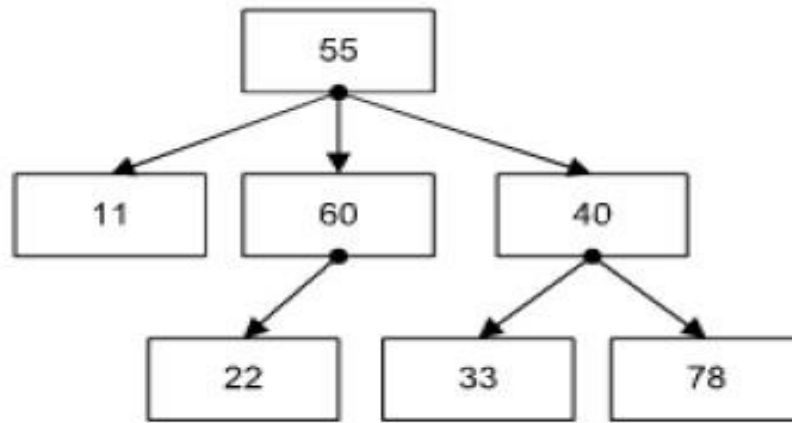
1. Aperçu

- Les structures arborescentes sont couramment utilisées dans de nombreuses applications informatiques telles que les **bases de données**, les **algorithmes de recherche**, **l'analyse de données**.
- Il existe plusieurs types de structures arborescentes, parmi lesquelles :
 - Les **arbres binaires** : chaque nœud a au plus deux enfants, appelés fils gauche et fils droit.
 - Les **arbres AVL** : ce sont des arbres binaires **équilibrés**, c'est-à-dire que la hauteur de chaque sous-arbre est équilibrée.
 - Les **arbres rouges et noirs** : ce sont également des arbres binaires **équilibrés**, mais leur équilibrage est obtenu en utilisant des règles de coloration des nœuds.
 - Les **arbres n-aires** : chaque nœud peut avoir jusqu'à n enfants.
 - Les **arbres de décision** : arbres utilisés pour la prise de décision en IA. Chaque nœud représente une question et les branches représentent les réponses possibles.
- Les structures d'arbre permettent de réaliser des opérations dynamiques, telles que **recherche**, **prédécesseur**, **successeur**, **minimum**, **maximum**, **insertion**, et **suppression** en temps proportionnel à la hauteur de l'arbre.
- La hauteur d'un arbre équilibré est en **$O(\log n)$** , n est le nombre de nœuds.

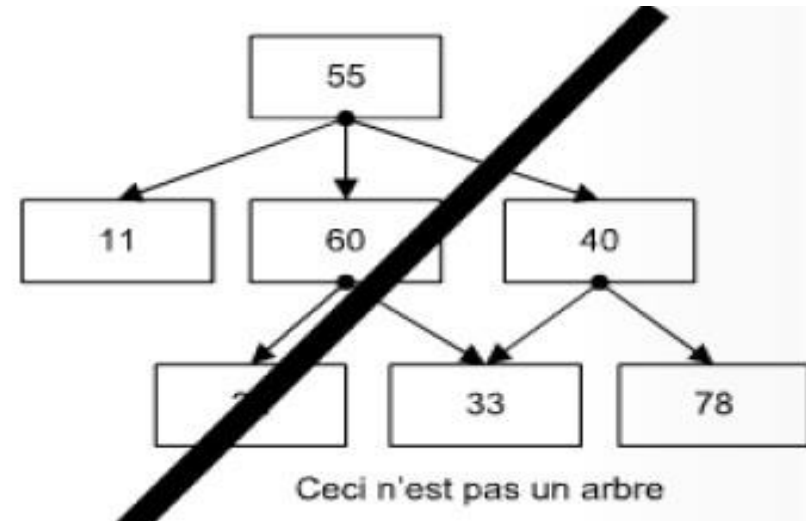
Introduction

2. Définitions

- ❖ Un **arbre** est une structure **composée** d'un ensemble de **noëuds** reliés entre eux selon des **liens Père-fils**
- ❖ Un arbre **N-aire** est un arbre où chaque nœud possède au **maximum N nœuds fils** (desendants)



Ceci est un arbre 3-aire



Ceci n'est pas un arbre

Introduction

3. Propriétés des arbres

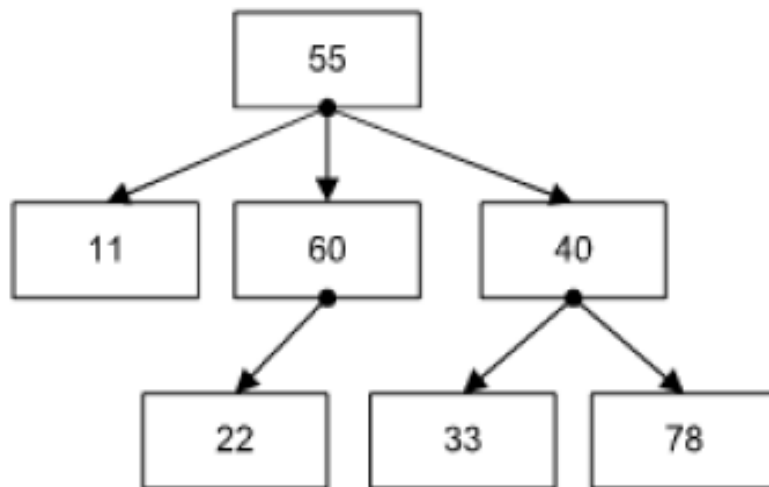
- ❖ Un **nœud**, appelé aussi **sommet**, contient une valeur (une clé) et indique les nœuds suivants
- ❖ Les **fil**s d'un nœud sont les **racines de ses sous-arbres**
- ❖ Une **feuille** d'un arbre est un **nœud sans fils** ; 11, 22, 33 et 78 sont des feuilles
- ❖ Une **branche** est un **chemin reliant le nœud racine à un nœud terminal (une feuille)**.
- ❖ La **hauteur d'un nœud** est la longueur **de la branche** le séparant de la feuille la plus éloignée.
- ❖ La **profondeur d'un nœud** mesure la distance entre la racine et le nœud, tandis que la hauteur mesure la distance entre le nœud et une feuille.
- ❖ La **hauteur d'un arbre** vaut la hauteur de la racine. La hauteur d'un arbre qui ne contient qu'un seul nœud (la racine) est généralement définie comme 0.
- ❖ Pour permettre un calcul général de la hauteur d'un arbre, la hauteur d'un arbre vide est définie comme -1

Introduction

4. Représentation d'un Arbre

■ Par un tableau

- ❖ Chaque nœud de **notre exemple** possède **au plus trois nœuds fils**
- ❖ On construit **un tableau à deux dimensions** où **chaque indice représente un nœud**
- ❖ Pour **connaître les suivants de chaque nœud**, il suffit de voir leur **indice dans les cases ad-hoc** du tableau
- ❖ **L'absence** d'un **nœud suivant** est représenté par la **valeur -1**



Numéro	[0]	[1]	[2]	[3]	[4]	[5]	[6]
Valeur	55	11	60	40	22	33	78
Suivant 1	1	-1	4	5	-1	-1	-1
Suivant 2	2	-1	-1	-1	-1	-1	-1
Suivant 3	3	-1	-1	6	-1	-1	-1

NB: Cette construction est compliquée à gérer, et de ce fait, elle n'est pas très utilisée.

Introduction

■ Par une liste chaînée

- Chaque nœud de notre exemple possède au plus trois nœuds fils
- Un nœud pouvant référencer jusqu'à 3 nœuds suivants : 3 attributs **gauche**, **milieu** et **droite** ou dans un tableau de 3 éléments
- Nous allons introduire la structure la suivante

```
typedef struct node {
```

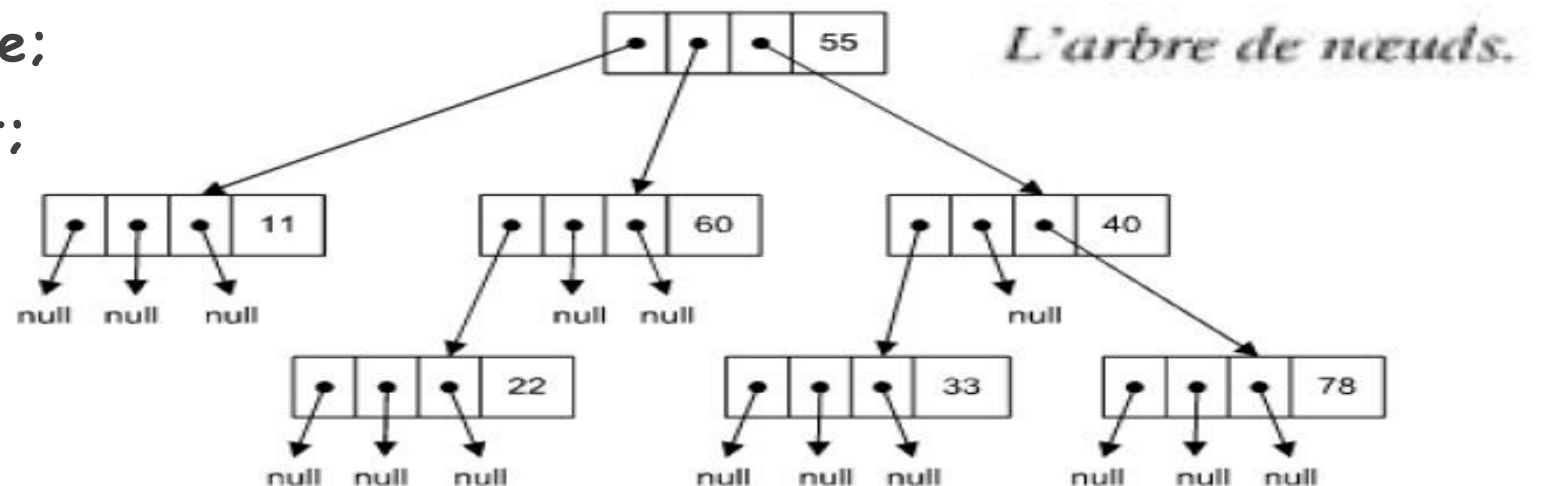
```
    Element data; // Element est le type de donnée que contient le noeud
```

```
    struct Nœud *left;
```

```
    struct Nœud *middle;
```

```
    struct Nœud *right;
```

```
} Node;
```





ARBRES BINAIRES

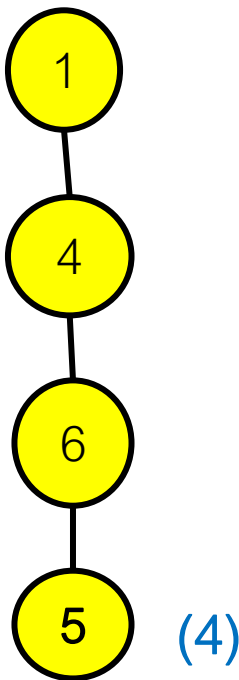
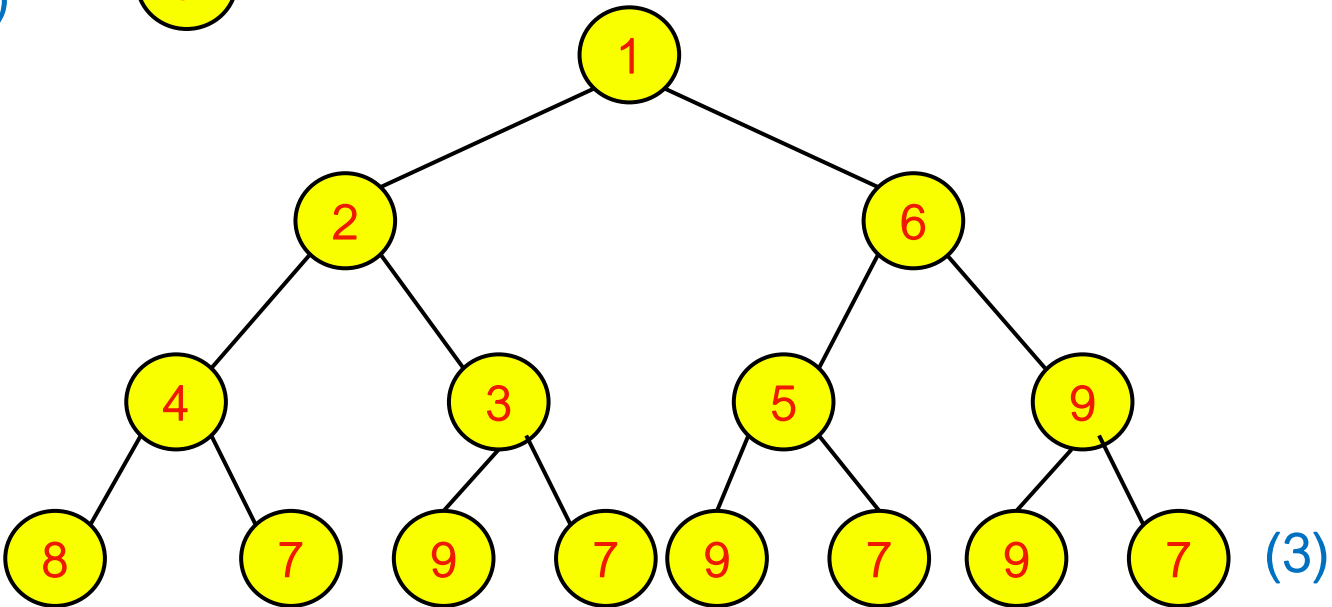
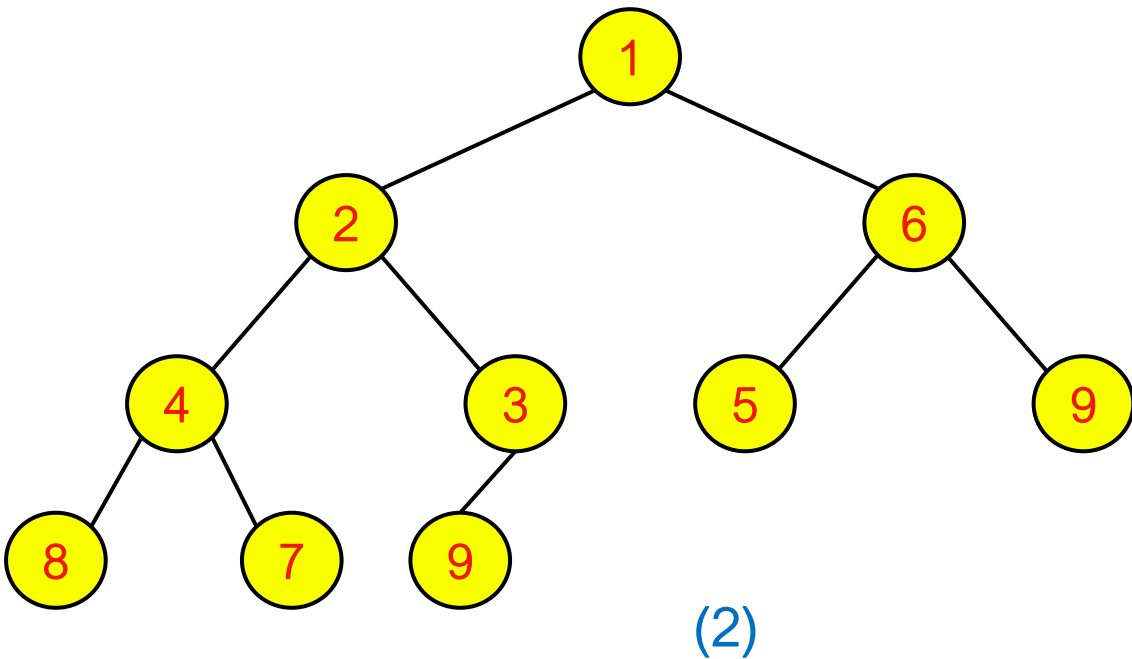
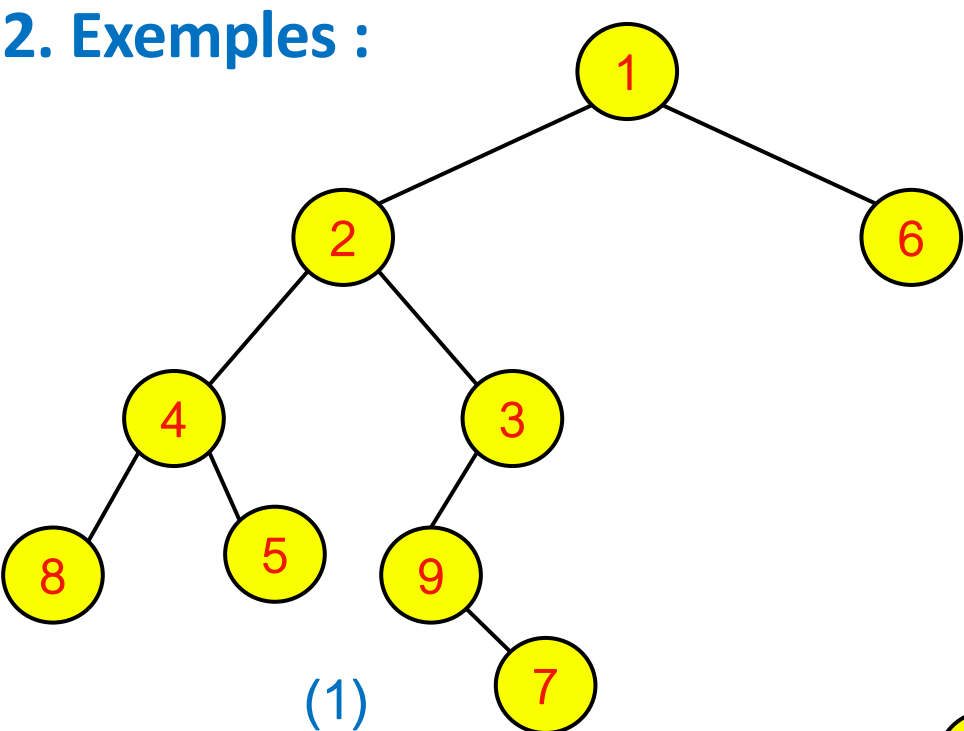
Arbres binaires

1. Définitions :

- Un **arbre binaire** est un arbre dont **chaque nœud ne peut avoir plus de deux fils**
- Un arbre binaire peut être soit **vide** ou bien composé d'un nœud **racine** et de deux sous-arbres binaires, appelés **sous-arbre gauche** et **sous-arbre droit**, qui sont eux-mêmes des arbres binaires.
- Un arbre binaire peut être :
 - (1) **Complet** : est un arbre binaire où **chaque niveau est rempli et le dernier niveau est rempli de gauche à droite**.
 - (2) **Parfait** : est un arbre binaire où **chaque niveau est rempli et le nombre de nœuds est une puissance de 2**.
 - (3) **Équilibré** : est un arbre binaire où **la hauteur de l'arbre est minimale**, c'est-à-dire que **la différence de hauteur** entre le sous-arbre gauche et le sous-arbre droit de chaque nœud **est au plus 1**.
 - (4) **Dégénéré** : est un arbre binaire dans lequel **chaque nœud n'a qu'un seul fils ou aucun fils**, ce qui signifie que l'arbre ressemble à une liste chaînée.

Arbres binaires

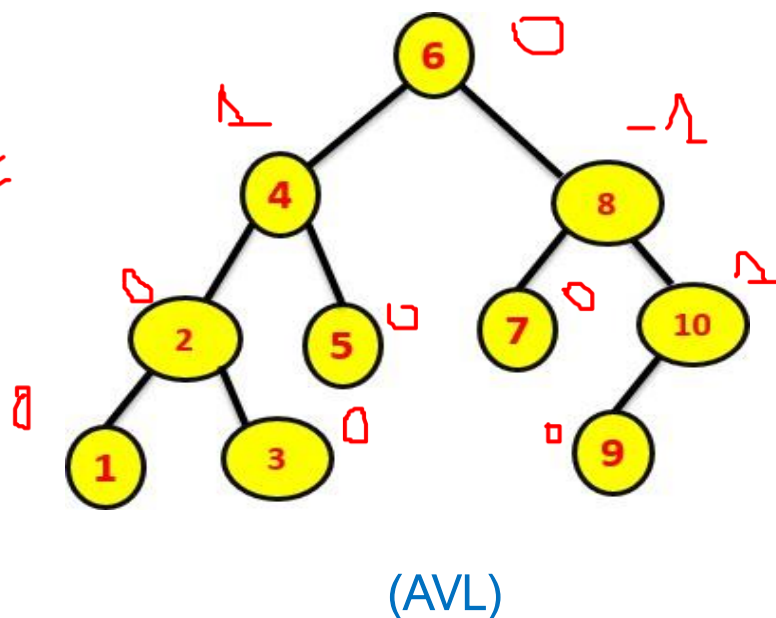
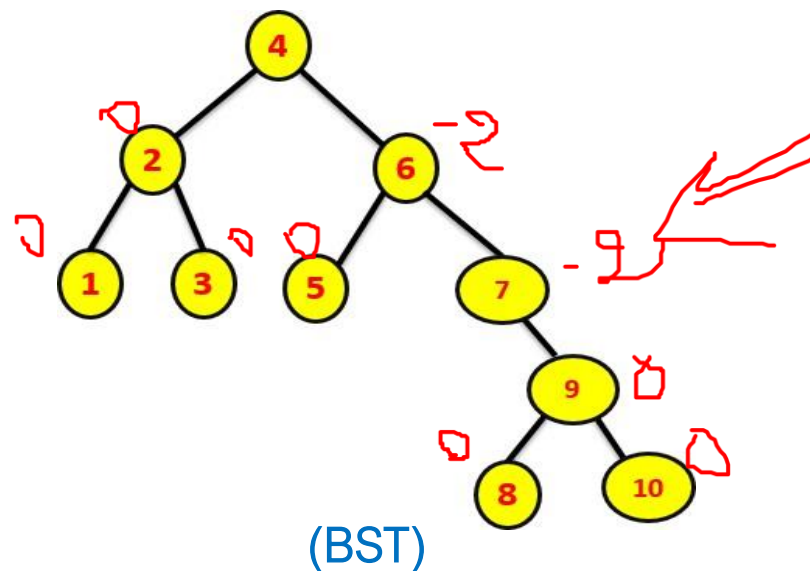
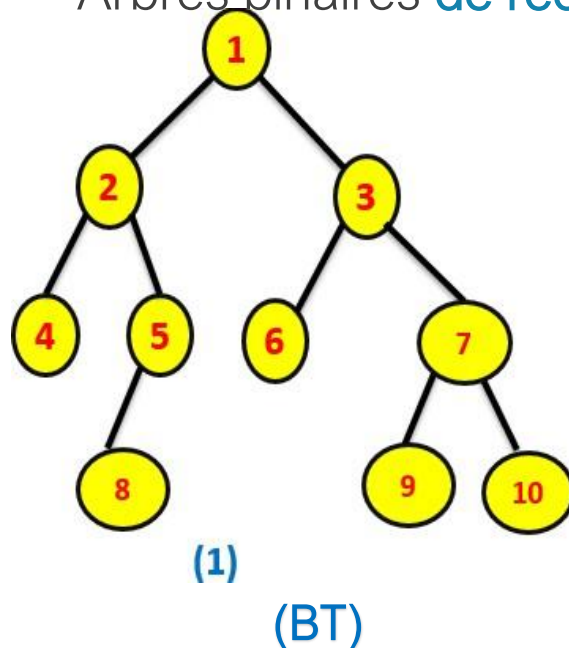
2. Exemples :



Arbres binaires

3. Types d'Arbres binaires

- Arbres binaires: Binary trees (BT)
- Arbres binaires **de recherche**: Binary Search Trees (BST)
- Arbres binaires **de recherche équilibrés** (AVL)



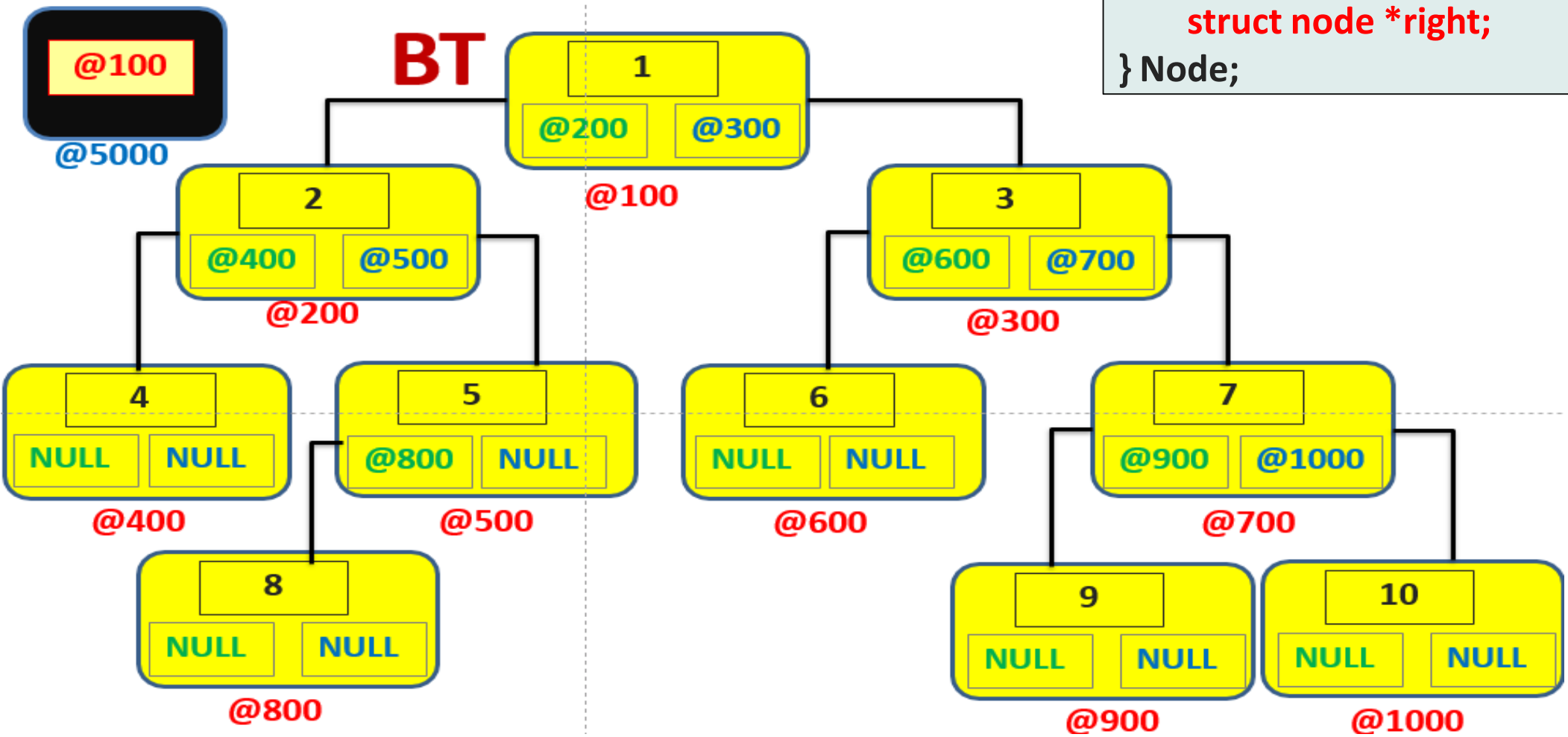
Autres types :

- Arbre de **syntaxe abstraite**,
- Arbres **d'expression**,
- Arbres **de Huffman**,
- Arborescence de **Arbres de jeux**, ...

Arbres binaires

Représentation en mémoire d'une arbre binaire

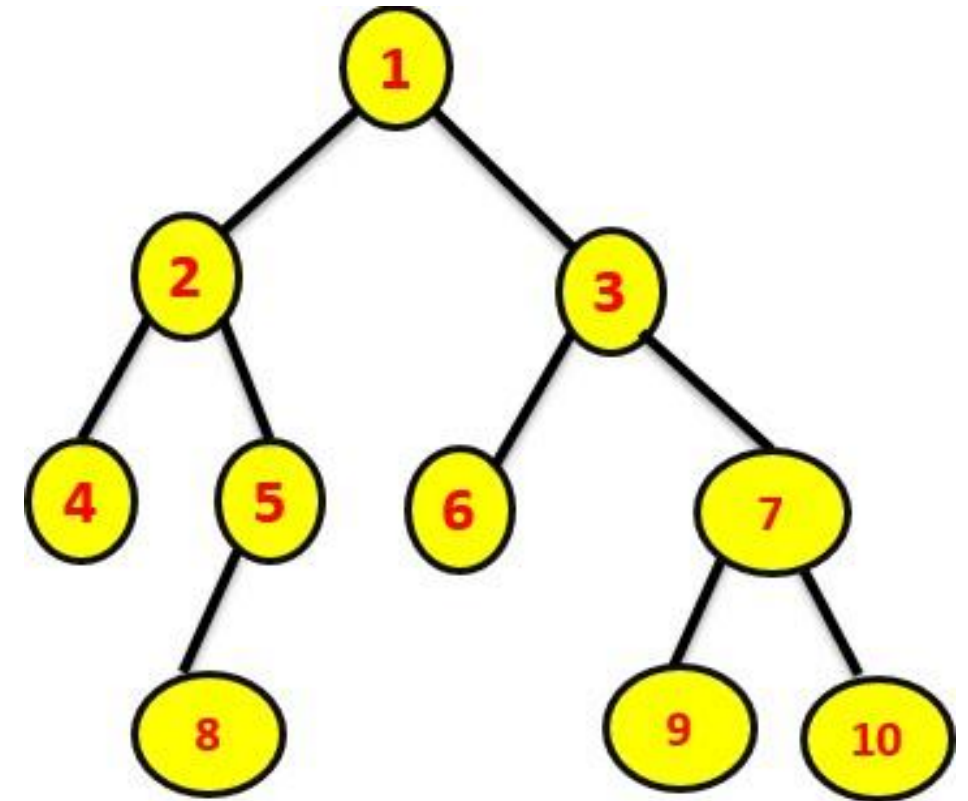
```
typedef struct node {  
    Element data;  
    struct node *left;  
    struct node *right;  
} Node;
```



Arbres binaires

4. Opérations de base sur les arbres binaires

- Création/initialisation de l'arbre,
- Vérification si l'arbre est vide,
- Lecture/Ecriture de la valeur en racine,
- Lecture du Sous arbre Gauche/Droit
- Ajout/insertion d'un nœud à l'arbre,
- Recherche d'un nœud,
- Recherche du nœud minimum,
- Suppression de tous les nœuds de l'arbre (vidage)
- Calcul de la taille d'un arbre binaire,
- Calcul du nombre de feuilles d'un arbre binaire,
- Affichage selon un parcours (infixe, postfixe, préfixe, en largeur)
- Vérification si un arbre est dégénéré,
- ...



Arbres binaires

■ Calcul de la taille d'un arbre binaire

```
int getSize(Node *root){  
    if(root == NULL)  
        return 0;  
    return 1+ getSize(root->left)+getSize(root->right);  
}
```

■ Nombre de feuilles d'un arbre binaire

```
int leafCount(Node *root){  
    if(root == NULL) return 0 ;  
    if(isLeaf(root) )return 1 ;  
    return leafCount(root->left)  
        + leafCount(root->right);  
}
```

Avec :

```
int isLeaf (Node *node){  
    return !node->left  
        && !node->right;  
}
```

Arbres binaires

■ Vérifier qu'un arbre n'est pas dégénéré :

```
int is_non_degenerate(Node *root){  
    if(root == NULL) return 0;  
    if((root->left != NULL ) && (root->right != NULL))return 1;  
    if(root->left == NULL) return is_non_degenerate(root->right);  
    return is_non_degenerate(root->left);  
}
```

■ Recherche par association dans un arbre binaire

```
int search(Node *root , Element val){  
    if(root == NULL) return 0 ;  
    if (root->data == val ) return 1;  
    if (search(root->left , val)) return 1;  
    return search(root->right,val) ;  
}
```


Arbres binaires

Parcours d'un arbre

Le parcours d'un arbre permet d'atteindre tous les nœuds d'un arbre pour appliquer des traitements sur ces nœuds.

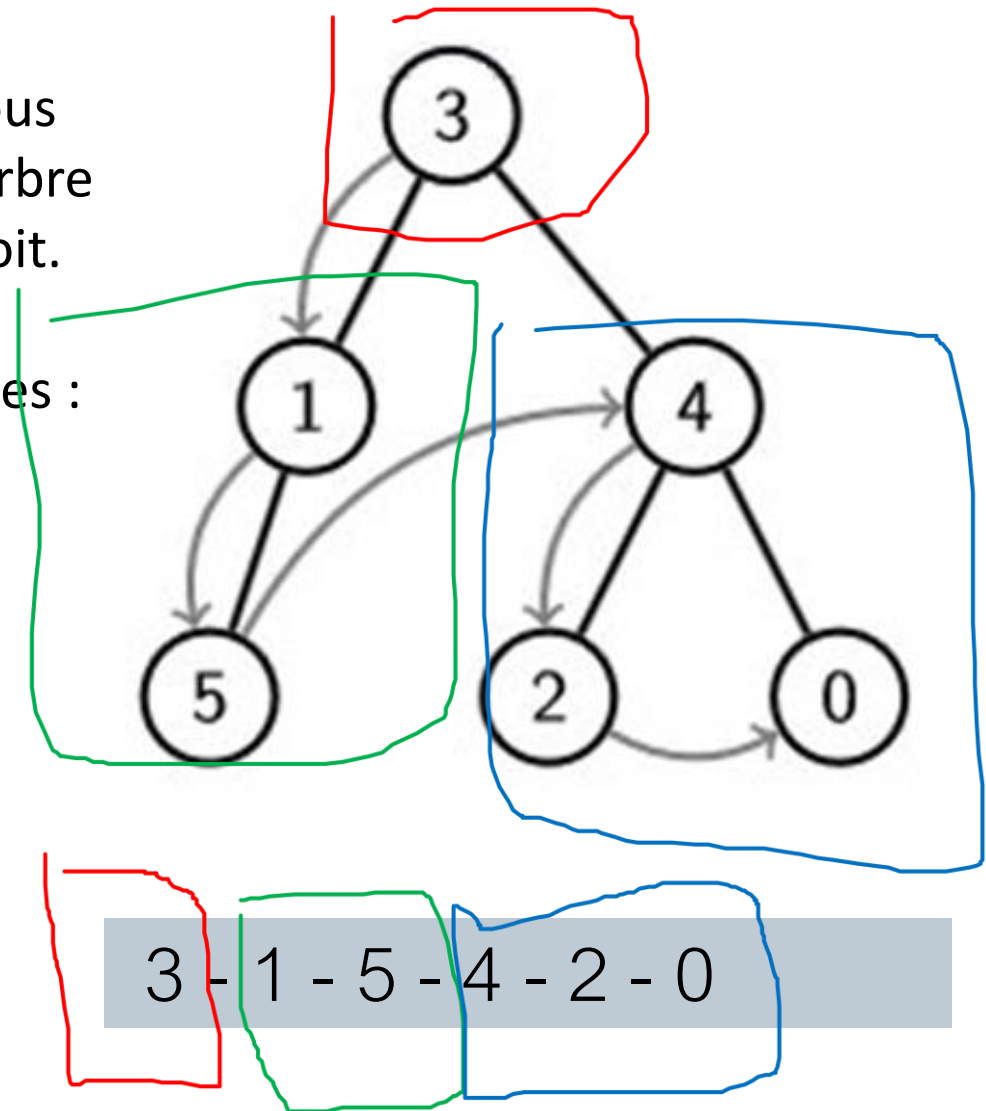
Nous distinguerons deux types de parcours :

- **Le parcours en profondeur (Depth traversal)** : permet d'explorer l'arbre en explorant jusqu'au bout une branche avant de passer à la suivante.
- **Le parcours en largeur (Breadth-First Traversal)** : permet d'explorer l'arbre niveau par niveau: parcourir tous les nœuds du niveau un puis ceux du niveau deux et ainsi de suite jusqu'à l'exploration de tous les nœuds.

Arbres binaires

■ Parcours préfixe

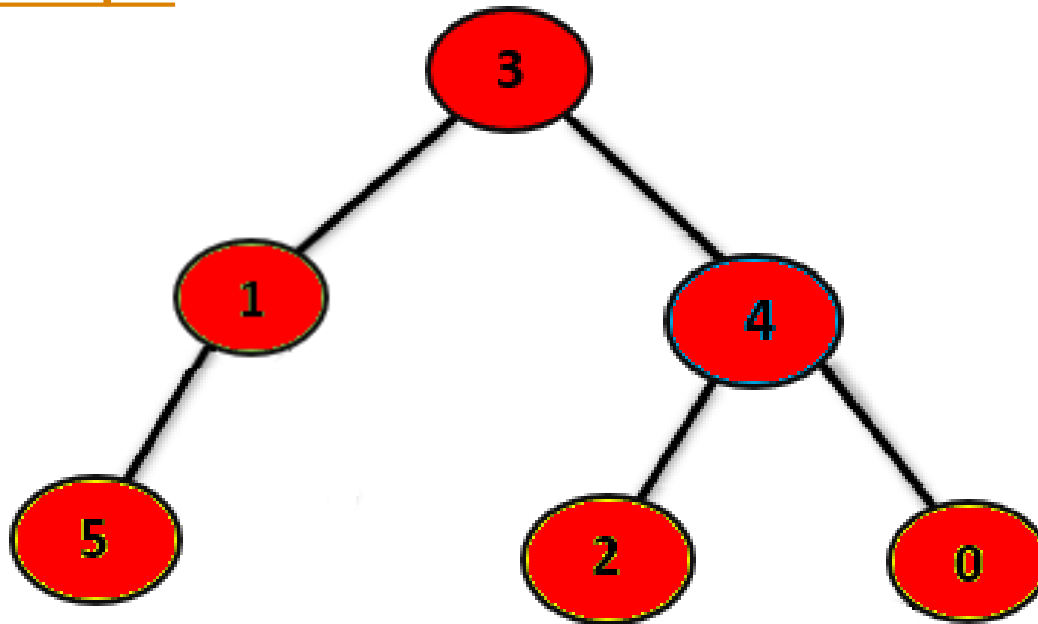
- On traite la racine de l'arbre, on parcourt tout le sous arbre de gauche, une fois qu'il n'y a plus de sous arbre gauche on parcourt les éléments du sous arbre droit.
- Ce type de parcours peut être résumé en trois lettres :
R G D (Racine Gauche Droit).



Arbres binaires

■ Parcours préfixe

Exemple



3 - 1 - 5 - 4 - 2 - 0

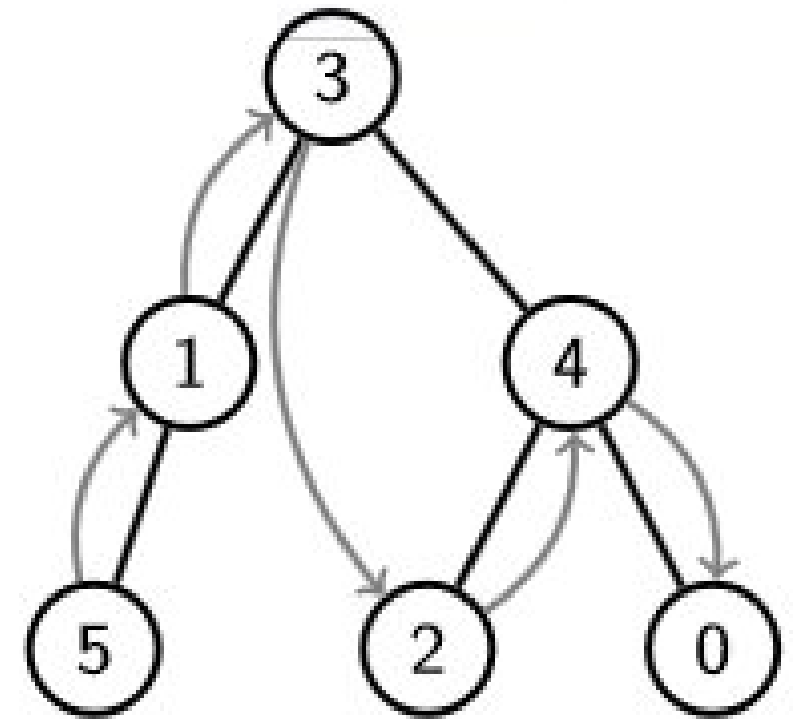
Code

```
void prefixe(Node *root)
{
    if(root)
    {
        printf ("%d - ",root->data);
        prefixe (root->left);
        prefixe (root->right);
    }
}
```

Arbres binaires

■ Parcours infixé

- On parcourt tout le sous arbre de gauche, une fois qu'il n'y a plus de sous arbre gauche, on traite la racine de l'arbre puis on parcourt les éléments du sous arbre droit.
- Ce type de parcours peut être résumé en trois lettres : **G R D** (Gauche Racine Droit).

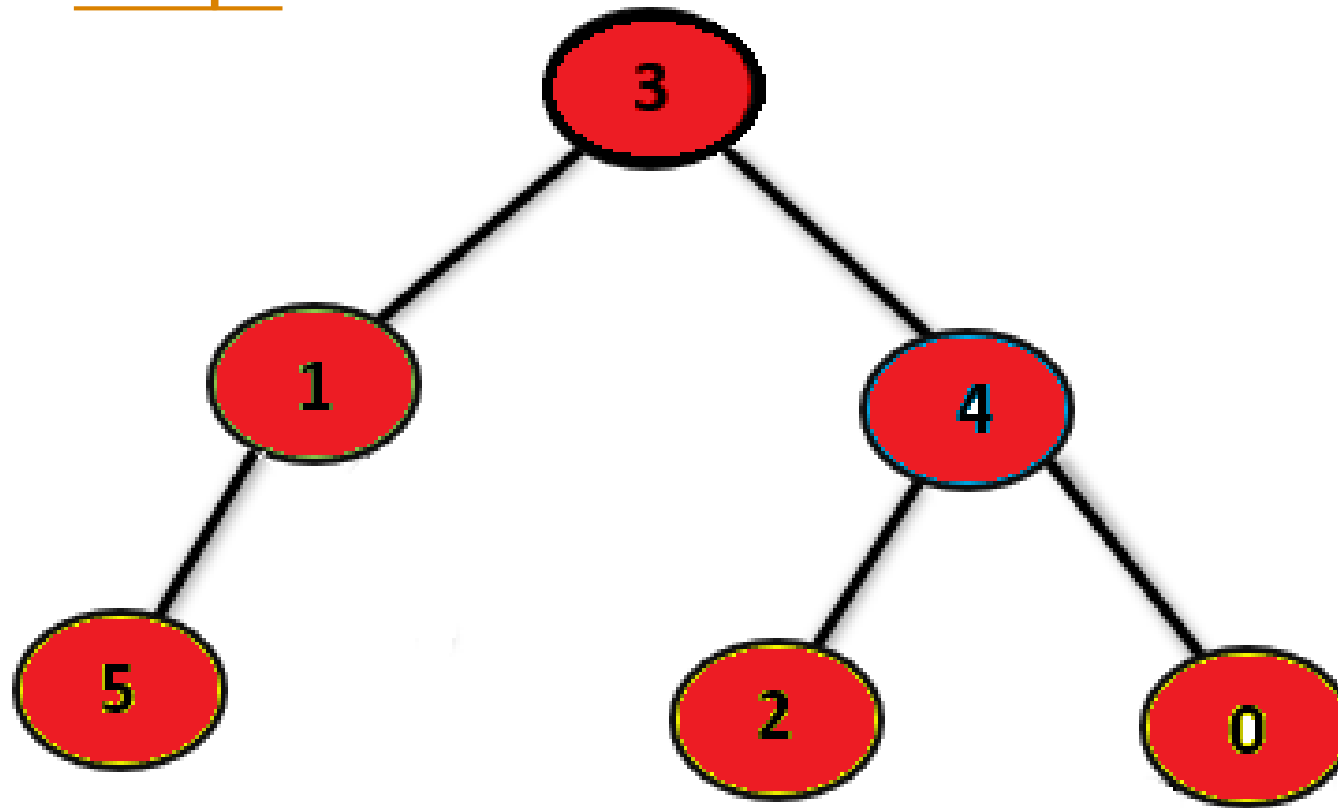


5 - 1 - 3 - 2 - 4 - 0

Arbres binaires

■ Parcours infixé

Exemple



5 - 1 - 3 - 2 - 4 - 0

Code

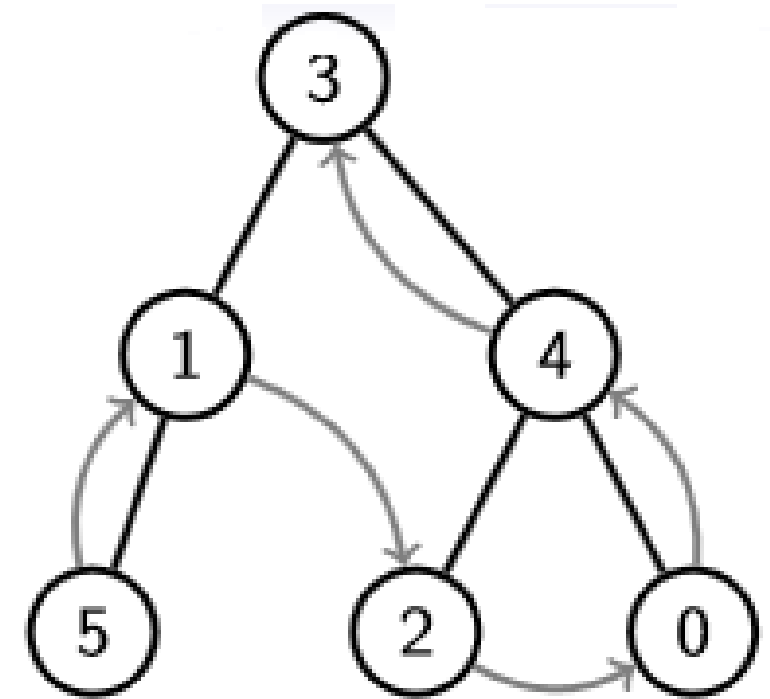
```
void infixe(Node *root)
{
    if(root)
    {
        infixe(root->left);
        printf("%d - ",root->data);
        infixe(root->right);
    }
}
```

Arbres binaires

■ Parcours postfixé

- On parcourt tout le sous arbre de gauche, une fois qu'il n'y a plus de sous arbre gauche, on parcourt les éléments du sous arbre droit puis on traite la racine de l'arbre.
- Ce type de parcours peut être résumé en trois lettres : **G D R** (Gauche Droit Racine).

```
void postfixe(Node *root)
{
    if(root)
    {
        postfixe(root->left);
        postfixe(root->right);
        printf("%d - ",root->data);
    }
}
```



5 - 1 - 2 - 0 - 4 - 3

■ Parcours en largeur

- Il s'agit d'un parcours dans lequel, on traite les nœuds un par un sur un même niveau. On passe ensuite sur le niveau suivant, et ainsi de suite.

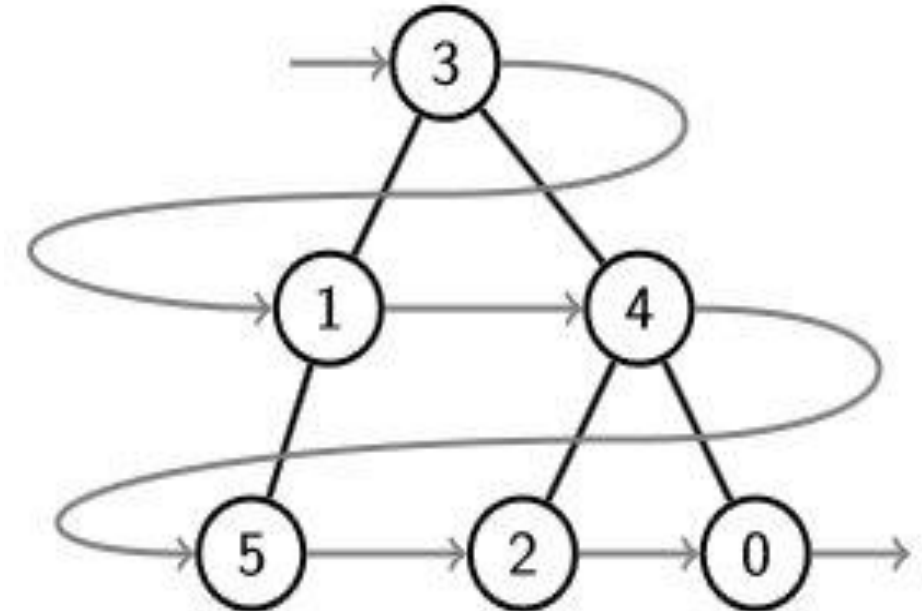
Code

```
void parcoursLargeur(Node *root)
{
    File F; // file of nodes
    Node *x ;
    enqueue(&F, root);
    while(!isEmpty(F) )
    {
        x = dequeue(&F);
        cout << x->data ;
        if(x->left != NULL)
            enqueue(&F, x->left);
        if(x->right != NULL)
            enqueue(&F, x->right);
    }
}
```

File



Exemple



3 - 1 - 4 - 5 - 2 - 0

Arbres binaires

Complexité des opérations sur les arbres :

Opération	Complexité
Calcul de la taille d'un arbre binaire	$O(n)$
Nombre de feuilles d'un arbre binaire	$O(n)$
Vérifier qu'un arbre n'est pas dégénéré	$O(n)$
Recherche par association dans un arbre binaire	$O(n)$
Recherche par association dans un arbre binaire ordonné	$O(h)$
Insertion dans un arbre binaire ordonné	$O(h)$

■ Implémentation Python

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BinaryTree:
    def __init__(self, root=None):
        self.root = root

    def get_size(self, root):
        if root is None:
            return 0
        return 1 + self.get_size(root.left) + self.get_size(root.right)

    def is_leaf(self, node):
        return not node.left and not node.right

    def leaf_count(self, root):
        if root is None:
            return 0
        if self.is_leaf(root):
            return 1
        return self.leaf_count(root.left) + self.leaf_count(root.right)
```

■ Implémentation Python

```
def is_non_degenerate(self, root):
    if root is None:
        return 0
    if root.left is not None and root.right is not None:
        return 1
    if root.left is None:
        return self.is_non_degenerate(root.right)
    return self.is_non_degenerate(root.left)

def search(self, root, val):
    if root is None:
        return 0
    if root.data == val:
        return 1
    if self.search(root.left, val):
        return 1
    return self.search(root.right, val)

def prefix(self, root):
    if root:
        print(root.data, end=" - ")
        self.prefix(root.left)
        self.prefix(root.right)
```

Arbres binaires

■ Implémentation Python

```
def infix(self, root):
    if root:
        self.infix(root.left)
        print(root.data, end=" - ")
        self.infix(root.right)

def postfix(self, root):
    if root:
        self.postfix(root.left)
        self.postfix(root.right)
        print(root.data, end=" - ")

def level_order_traversal(self, root):
    queue = [root]
    while queue:
        current = queue.pop(0)
        print(current.data, end=" ")
        if current.left:
            queue.append(current.left)
        if current.right:
            queue.append(current.right)
```



ARBRES BINAIRES DE RECHERCHE

Arbres binaires de recherche

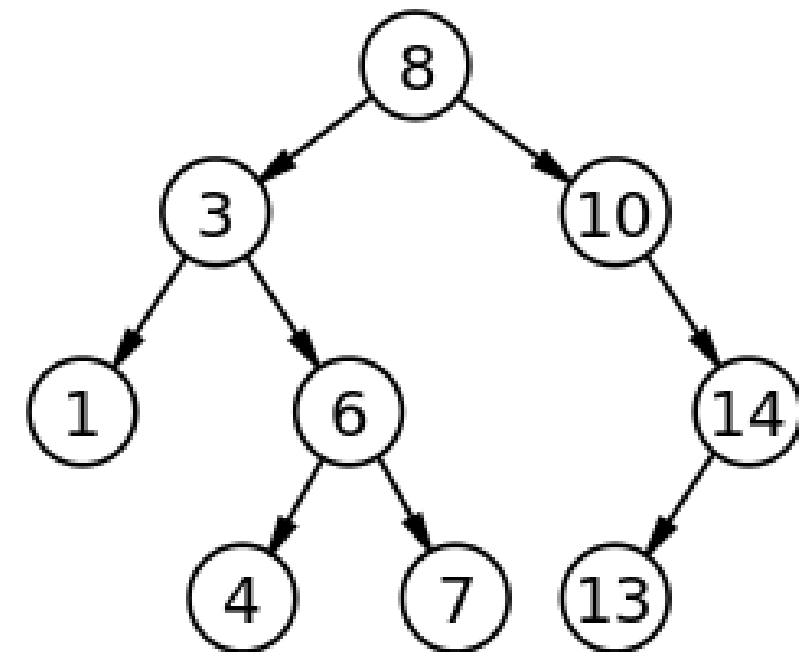
1. Définitions

Un arbre binaire de recherche est arbre binaire tel que pour chaque nœud de valeur x , toutes les valeurs des nœuds de son sous-arbre gauche sont strictement inférieures à x , et toutes les valeurs des nœuds de son sous-arbre droit sont strictement supérieures à x .

2. Propriétés des ABR

- Le parcours infixe (G-R-D) ordonne les nœuds par contenu **croissant**.
- Le parcours infixe (D-R-G) ordonne les nœuds par contenu **décroissant**.
- Si un nœud a un fils gauche, son **prédécesseur** est le nœud le plus à droite dans son sous-arbre gauche. Ce prédécesseur n'a pas de fils droit.
- Si un nœud a un fils droit, son **successeur** est le nœud le plus à gauche dans son sous-arbre droit. Ce successeur n'a pas de fils gauche.

Exemple



Arbres binaires de recherche

3. Operations sur ABR

- Initialiser l'arbre
- Créer un élément
- Vérifier si un arbre est vide
- Parcours de l'arbre
- Recherche d'un élément,
- Insertion d'un nouveau élément,
- Suppression,
- Successeur,
- Prédecesseur,
- Minimum,
- Maximum,
- ...

■ Recherche par association dans un arbre binaire de recherche

```
int exist(Node *root , Element val){  
    if(root == NULL)return 0;  
    if (root->data < val )  
        return exist(root->right ,val) ;  
    if(root->data > val )  
        return exist(root->left,val) ;  
    return 1;  
}
```

L'arbre de recherche =>

On peut choisir le sous-arbre
dans lequel il faut rechercher val

Coût :

- Meilleur des pires cas?
- Pire des pires cas?
- f (profondeur)?

```
Node** find(Node **ptr_root , Element val){  
    if(*ptr_root == NULL)return NULL;  
    if ((*ptr_root)->data < val ) return find(&(*ptr_root)->right ,val) ;  
    if((*ptr_root)->data > val ) return find(&(*ptr_root)->left,val) ;  
    return ptr_root;  
}
```

- Recherche par association dans un arbre binaire de recherche

Solution itérative

```
int exist(Node * root, int val){  
    while (root != NULL && root->data != val)  
        if (val < root->data) root = root->left;  
        else root = root->right;  
    return root!=NULL;  
}
```

```
Node **find(Node **ptr_root, int val){  
    while (*ptr_root != NULL && (*ptr_root)->data != val)  
        if (val < (*ptr_root)->data) ptr_root = &(*ptr_root)->left;  
        else ptr_root = &(*ptr_root)->right;  
    return *ptr_root!=NULL?ptr_root:NULL;  
}
```

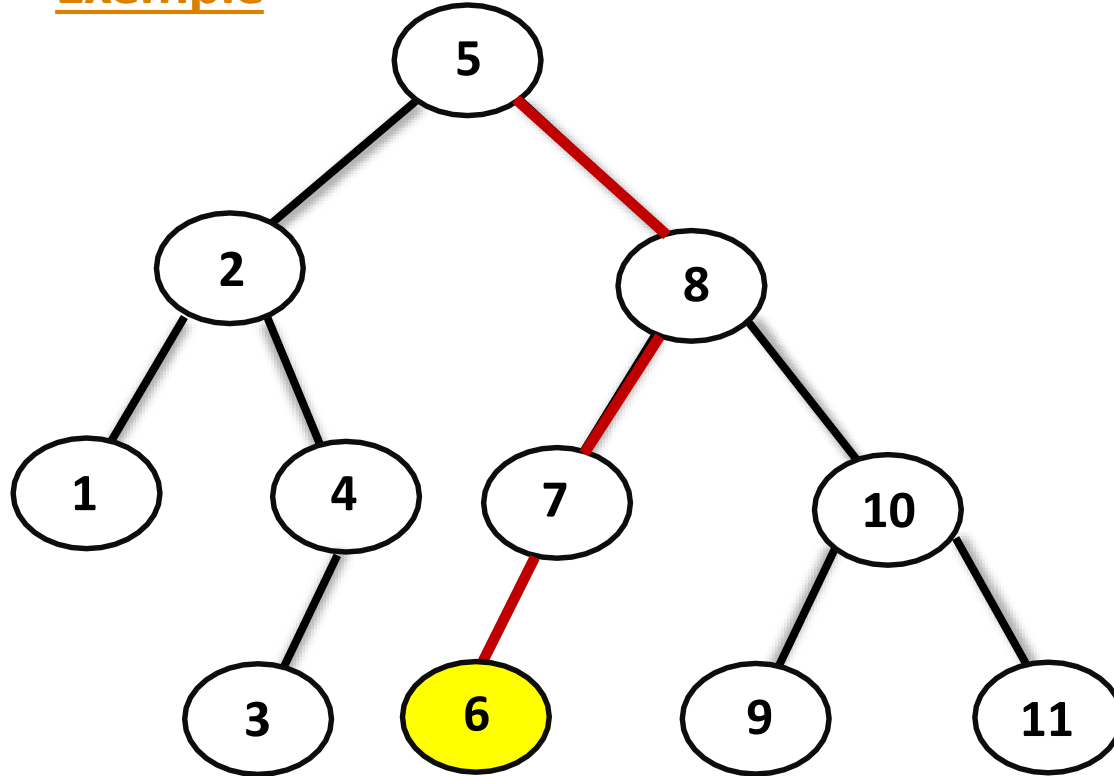

Arbres binaires de recherche

■ Insertion dans un arbre binaire de recherche

Concept:

- Descendre l'arborescence comme pour Rechercher,
- Si la nouvelle clé n'est pas trouvée, insérez un nouveau nœud au dernier endroit parcouru

Exemple



Pour insérer un nouveau élément :

- Créer le nouveau élément

6

- Si l'arbre est vide

Le nouveau élément est le racine

- Si l'arbre est plein

Trouver la position selon sa valeur

Arbres binaires de recherche

■ Insertion dans un arbre binaire de recherche

Code

```
void insert(Node **ptr_root , Element val)
{
    if(*ptr_root == NULL) *ptr_root=createLeaf( val);
    else
        if( val > (*ptr_root)->data)
            insert(&((*ptr_root)->right ),val);
        else
            if(val<(*ptr_root)->data)
                insert(&((*ptr_root)->left) , val) ;
            else
                ;// Gestion du cas de duplication si besoin
}
```

Avec:

```
Node * createLeaf (Element val )
{
    Node *tmp;
    tmp=(Node *)malloc(sizeof(Node));
    tmp->data =val;
    tmp->right= tmp->left=NULL;
    return tmp;
}
```

Arbres binaires de recherche

■ Insertion dans un arbre binaire de recherche - version 2

```
Node * insert(Node *root , Element val){  
    if(root == NULL) return createLeaf( val);  
    else  
        if( val > root->data)  
            root->right =insert(root->right ,val);  
        else  
            if( val < root->data)  
                root->left = insert(root->left , val);  
            else  
                ; // Gestion du cas de duplication si besoin  
    return root;  
}
```

Attention :

Pour insérer une valeur x dans un arbre a, l'appel à cette fonction doit toujours se faire par :

```
a=insert(a , x);
```

■ Noeud Successeur

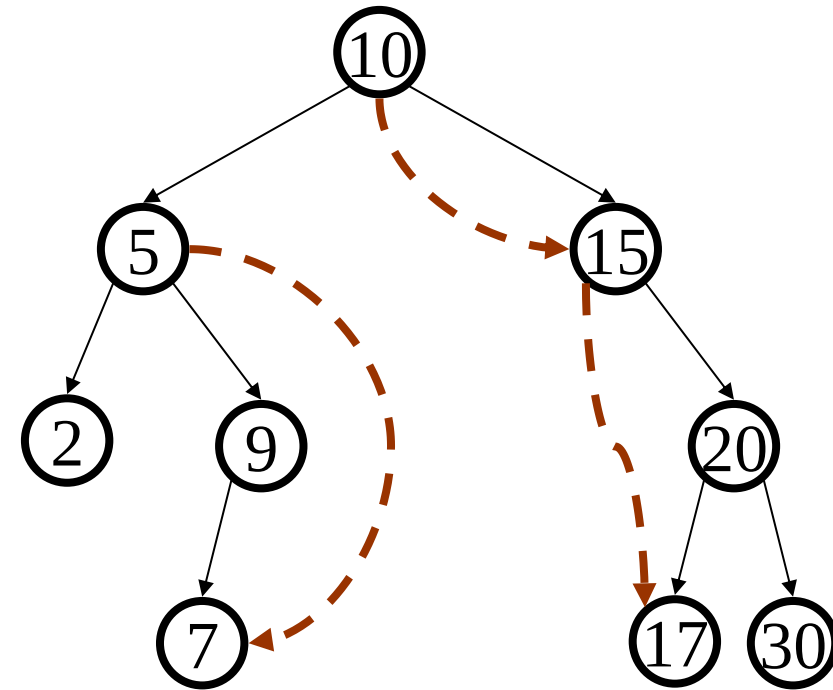
Noeud plus grand suivant dans la sous-arborescence de ce noeud

```
Node * succ(Node * root) {  
    if (!root || !root->right)  
        return NULL;  
    else  
        return min(root->right);  
}
```

Rechercher **minimum**

```
Node * min(Node * root) {  
    if(!root) return NULL;  
    while(root->left) root=root->left;  
    return root;}  

```



Combien d'enfants le successeur d'un noeud peut-il avoir?

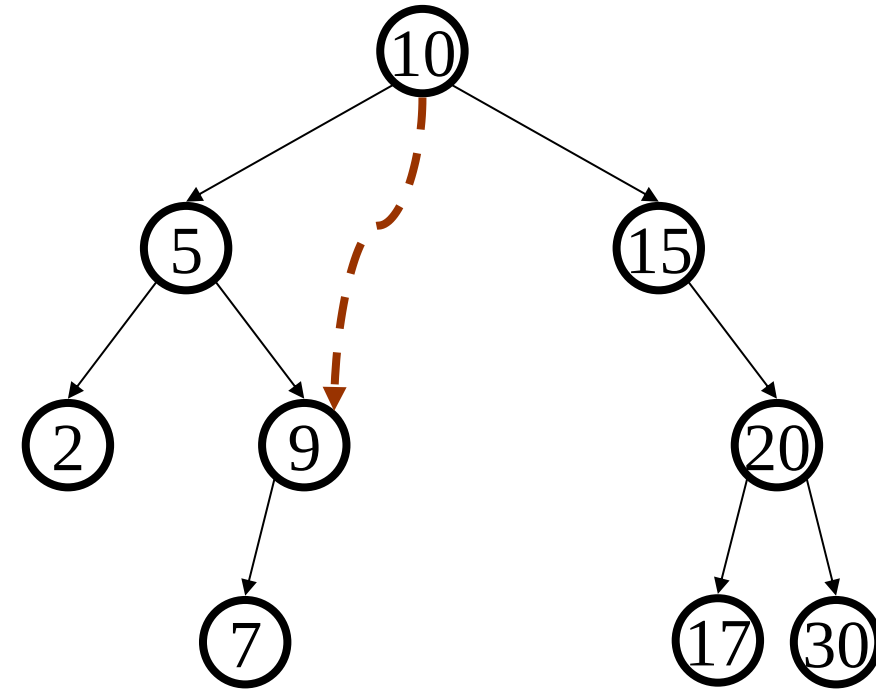
■ Noeud Prédécesseur

Noeud plus petit suivant dans la sous-arborescence de ce noeud

```
Node * pred(Node * root) {  
    if (!root || !root->left)  
        return NULL;  
    else  
        return max(root->left);  
}
```

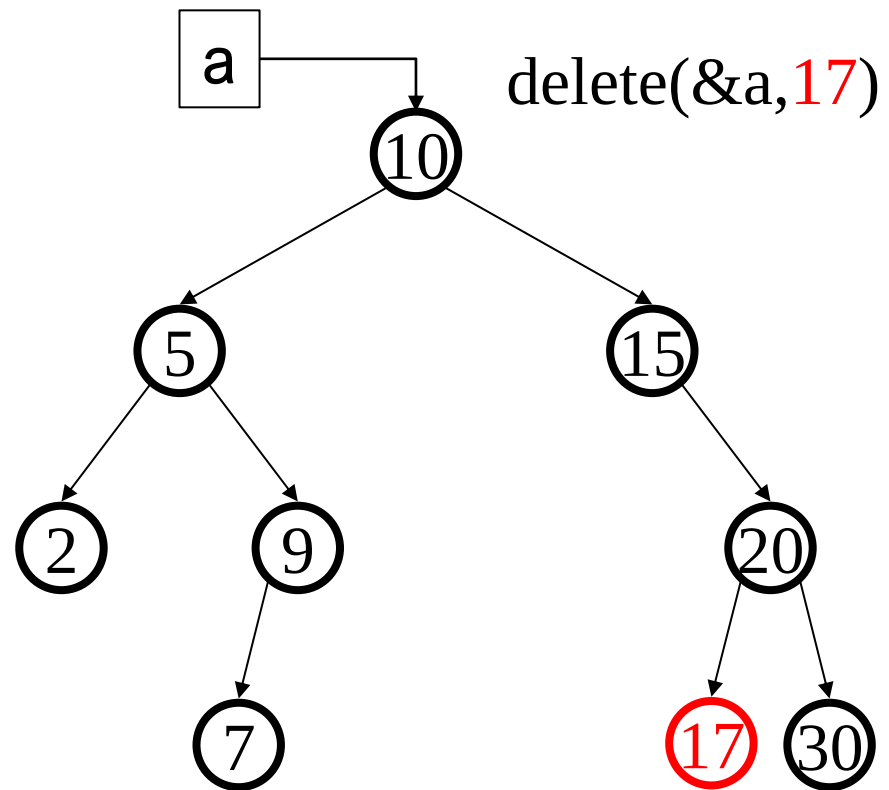
Rechercher **maximum**

```
Node * max(Node * root) {  
    if (!root) return NULL;  
    while (root->right) root = root->right;  
    return root;  
}
```

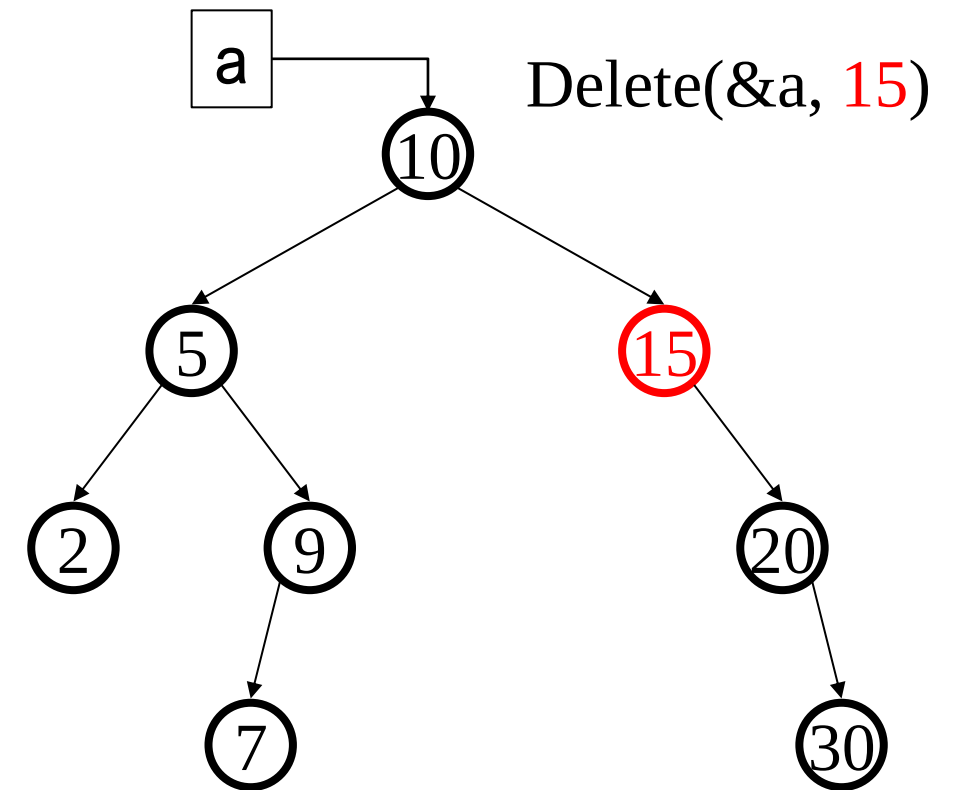


■ Suppression

1er cas : Suppression d'une feuille



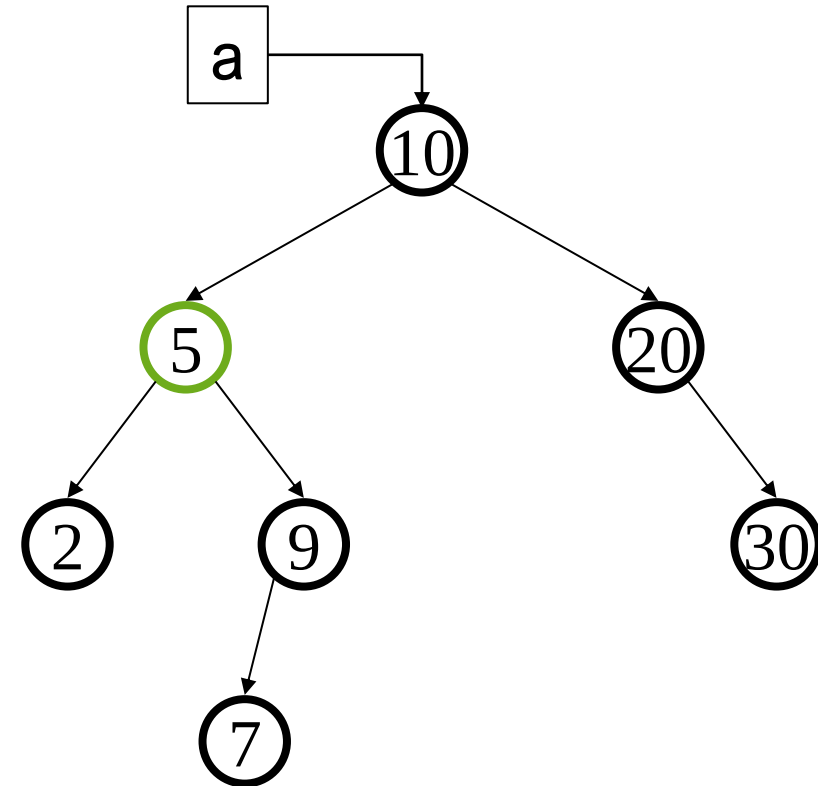
2ème cas : Suppression d'un noeud à un seul fils



3ème cas : Suppression d'un noeud
à deux fils

Remplacer le nœud par le descendant dont la valeur est garantie entre
les sous-arbres gauche et droit : le successeur

Delete(&a, 5)



Aurions-nous pu utiliser prédécesseur à la place?

Arbres binaires de recherche

Code

```
void Delete(Node ** ptr_root , int val) {
    Node ** tmp = find(ptr_root,val),*toDelete=*tmp;
    if (tmp) {
        if (!(*tmp)->left) {      // Leaf or one child
            *tmp = (*tmp)->right;
            free(toDelete);
        } else
            if (!(*tmp)->right) { // One child
                *tmp = (*tmp)->left;
                free(toDelete);
            } else {              // Two children
                Node* successor = succ(*tmp);
                (*tmp)->data = successor->data;
                Delete(&(*tmp)->right, successor->data);
            }
    }
}
```


Complexité des différentes opérations :

Opération	Complexité
Recherche par association dans un arbre binaire de recherche	$O(h)$
Insertion dans un arbre binaire de recherche	$O(h)$
suppression dans un arbre binaire de recherche	$O(h)$



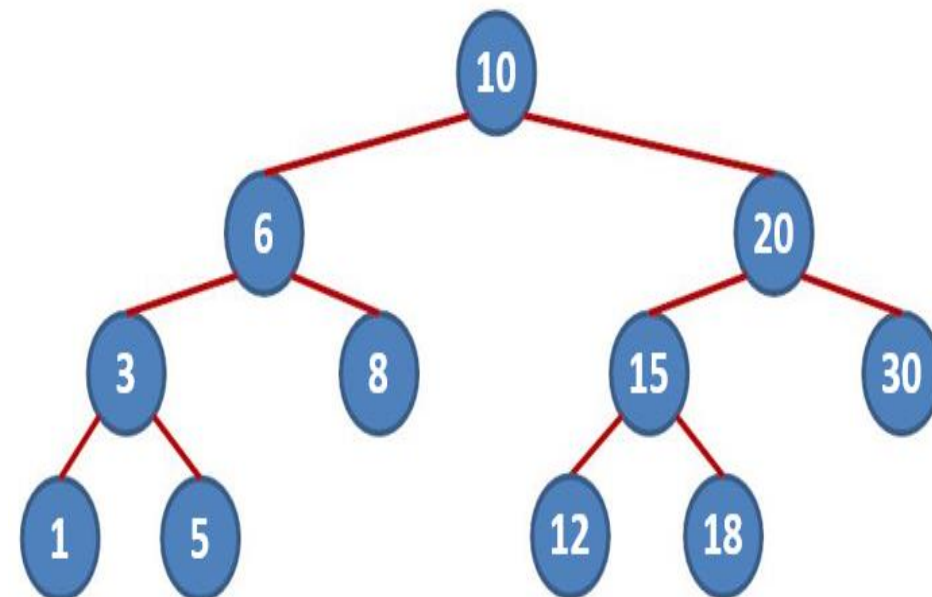
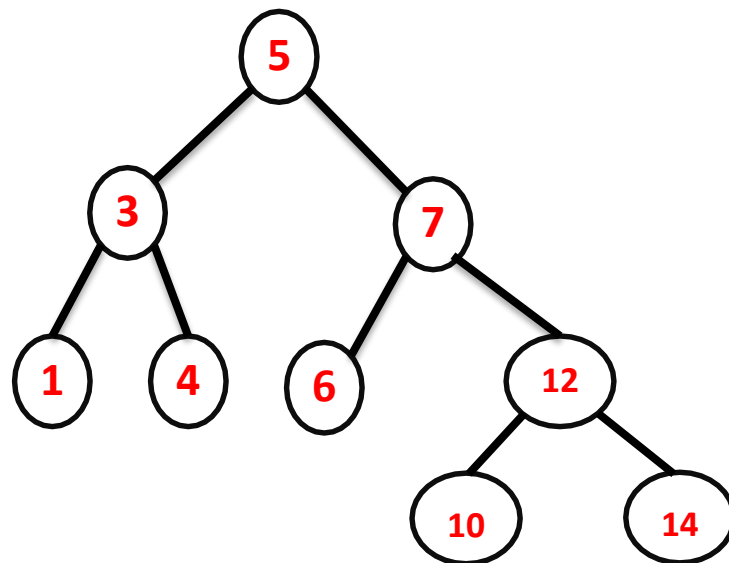
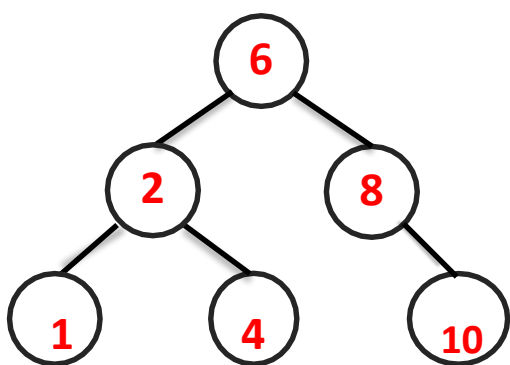
ARBRES BINAIRES DE RECHERCHE EQUILIBRES

Arbres binaires de recherche équilibrés

1. Définitions

- AVL (Adelson-Velskii et Landis) est un Arbre binaire de recherche **équilibré**.
- pour chaque nœud d'un arbre AVL, **la différence de hauteur de ses sous-arbres est au maximum un ($|Hauteur(fg) - Hauteur(fd)| \leq 1$)**
- Cette propriété garantie que la hauteur d'un arbre AVL est en **$O(\log n)$** , où n est le nombre de nœuds
- Pour contrôler l'équilibrage, il faut ajouter un champ **balance** (facteur d'équilibrage) ou calculer la **hauteur** au niveau de chaque nœud
- L'intérêt des arbres AVL est d'optimiser la recherche

Exemples



2. Propriétés d'un arbre AVL

- Pour un un arbre AVL ayant n sommets et de hauteur h , on a :

$$\log_2 (1+n) \leq 1+ h \leq \alpha \log_2 (2 + n) \text{ avec } \alpha \leq 1,44.$$

$$n \leq 2^{h+1} - 1 \text{ donc } \log_2(1+n) \leq 1+ h$$

Par exemple, si $n = 100000$, $17 \leq h \leq 25$.

- la **profondeur maximale** d'un arbre binaire équilibré est $1.44 * \log_2 n$
- La recherche dans un tel arbre n'exige jamais plus de 44% de plus de comparaisons que pour un arbre binaire complet

Arbres binaires de recherche équilibrés

3. Les opérations sur un arbre AVL

- Insertion,
- Suppression,
- Recherche,
- Parcours en infixe, préfixé, postfixé, en largeur

Remarque:

Les opérations d'insertion et de suppression nécessitent d'équilibrer l'arbre pour qu'il garde la propriété d'un AVL

Type du nœud d'un arbre AVL

```
typedef struct node{  
    int data;  
    struct node *left, *right;  
    int height=0; // int balance=0;  
} Node;
```

4. Equilibrer un arbre AVL

Après insertion, si l'arbre devient non équilibré, l'équilibrer

- L'insertion d'un nœud est similaire à l'insertion dans un arbre binaire de recherche en gardant l'équilibre.
- À chaque insertion, on remonte du nouveau nœud jusqu'à la racine de l'arbre en calculant la **différence de hauteur** des sous-arbres de chacun des nœud rencontrés.
- Si cette différence est égale à +2 ou -2, on rééquilibre avec la bonne rotation.
 - Rotation simple à gauche
 - Rotation simple à droite
 - Rotation double droite gauche
 - Rotation double gauche droite
- Pour rééquilibrer un arbre AVL après une insertion, **une seule rotation simple ou double suffit.**
- Pour rééquilibrer un arbre AVL après une suppression, il faut **jusqu'à h rotations ou double rotations** (h est la hauteur de l'arbre)

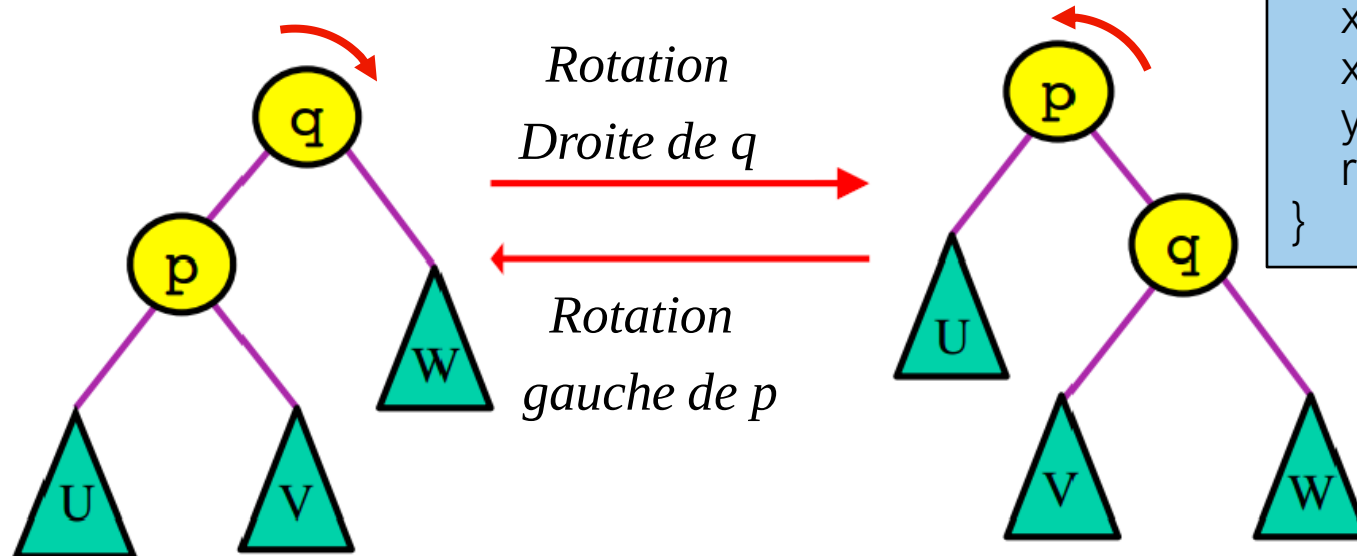
Arbres binaires de recherche équilibrés

Opérations de rotation

Les rotations gauche et droite transforment un arbre :

- Elles préservent l'ordre infixe
- Elles se réalisent en temps constant.

Rotation simple



```
Node* rightRotate(Node* y) {
    Node *x = y->left, *t = x->right;
    x->right = y;
    y->left = t;
    y->height = 1 + max(height(y->left), height(y->right));
    x->height = 1 + max(height(x->left), height(x->right));
    return x;
}
```

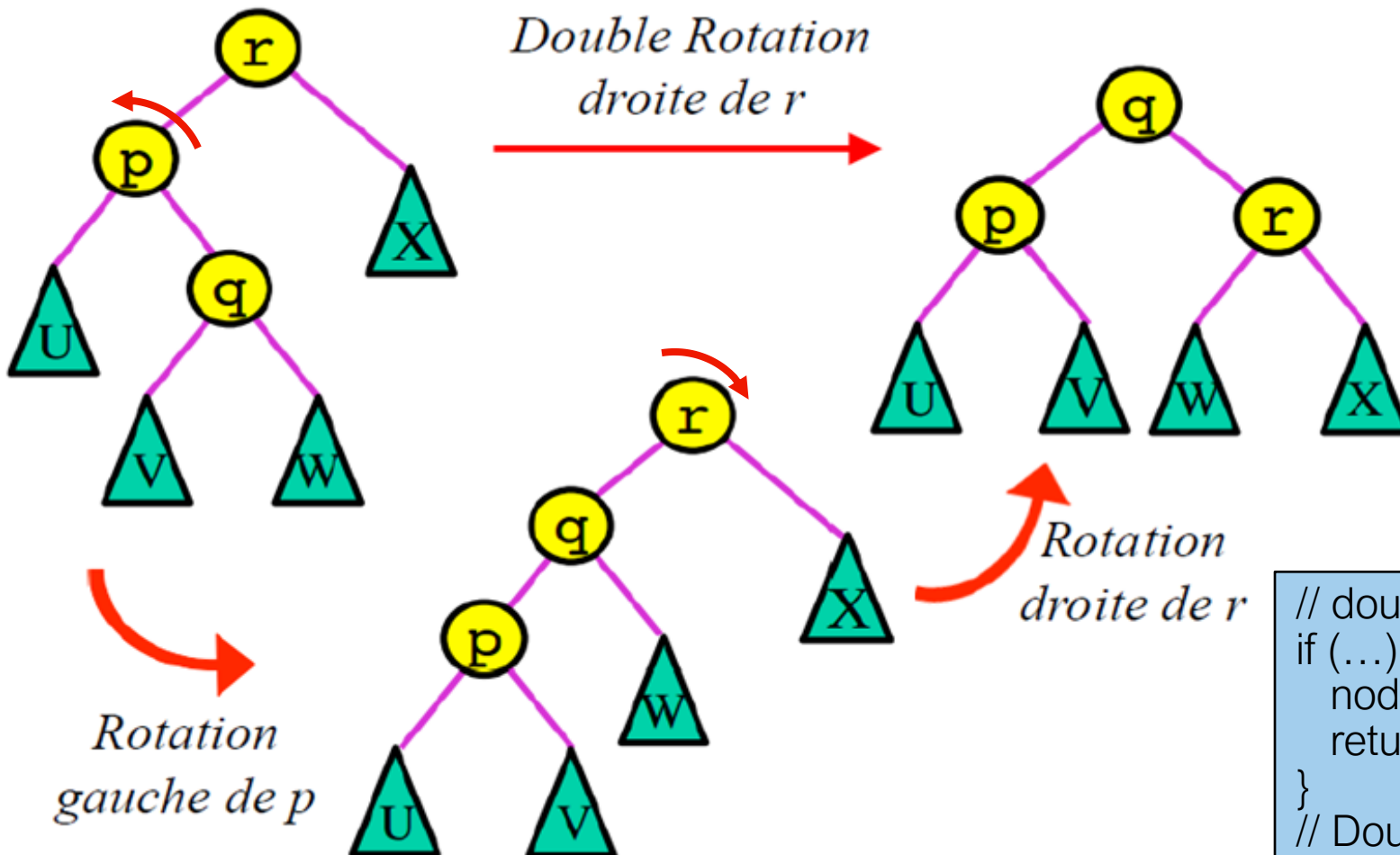
```
Node* leftRotate(Node* x) {
    Node *y = x->right, *t = y->left;
    y->left = x;
    x->right = t;
    x->height = 1 + max(height(x->left), height(x->right));
    y->height = 1 + max(height(y->left), height(y->right));
    return y;
}
```

```
int height(Node* node) {
    if (node == NULL) return -1;
    return node->height;
}
```

```
int max(int a, int b) {
    return (a > b) ? a : b;
}
```

Arbres binaires de recherche équilibrés

Double rotation :



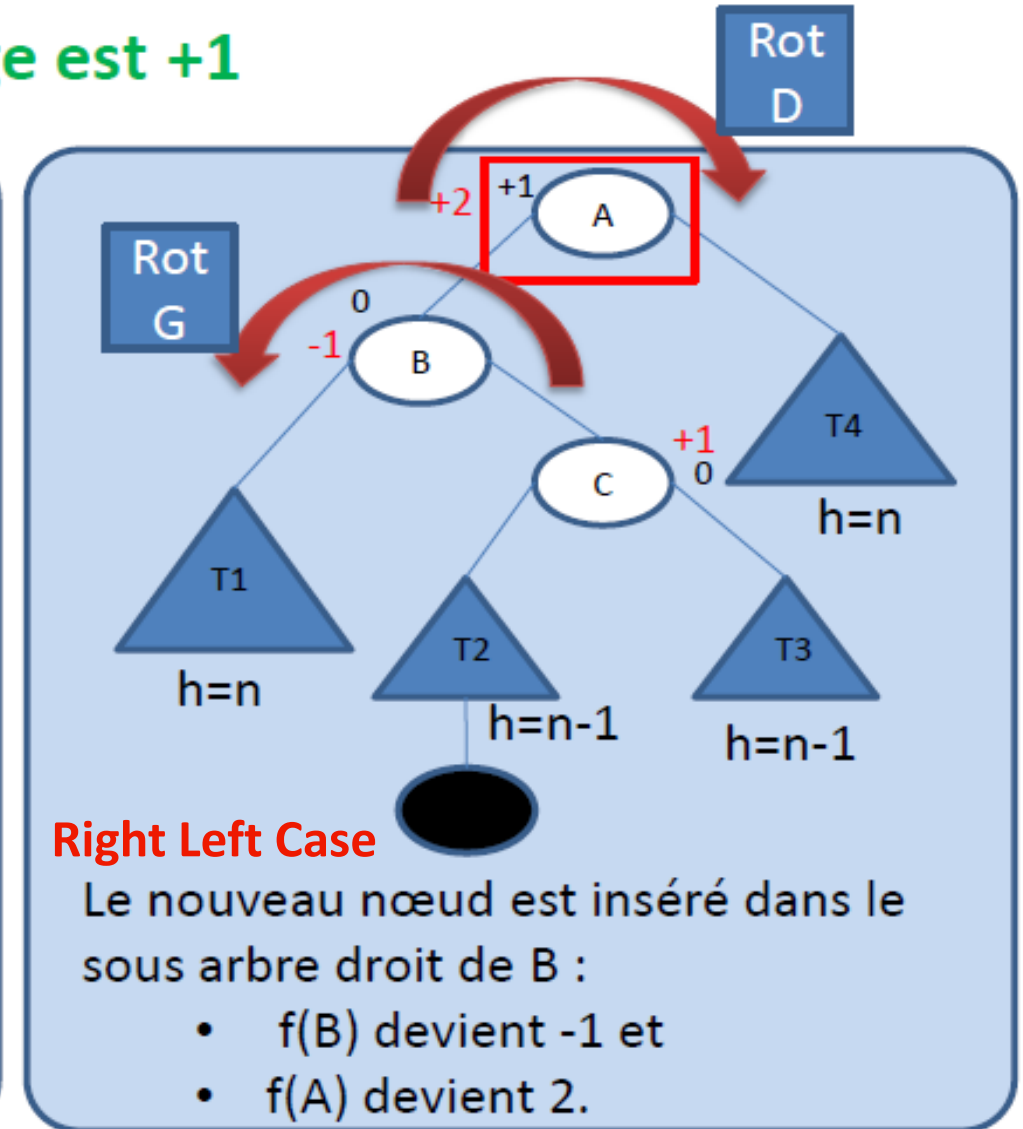
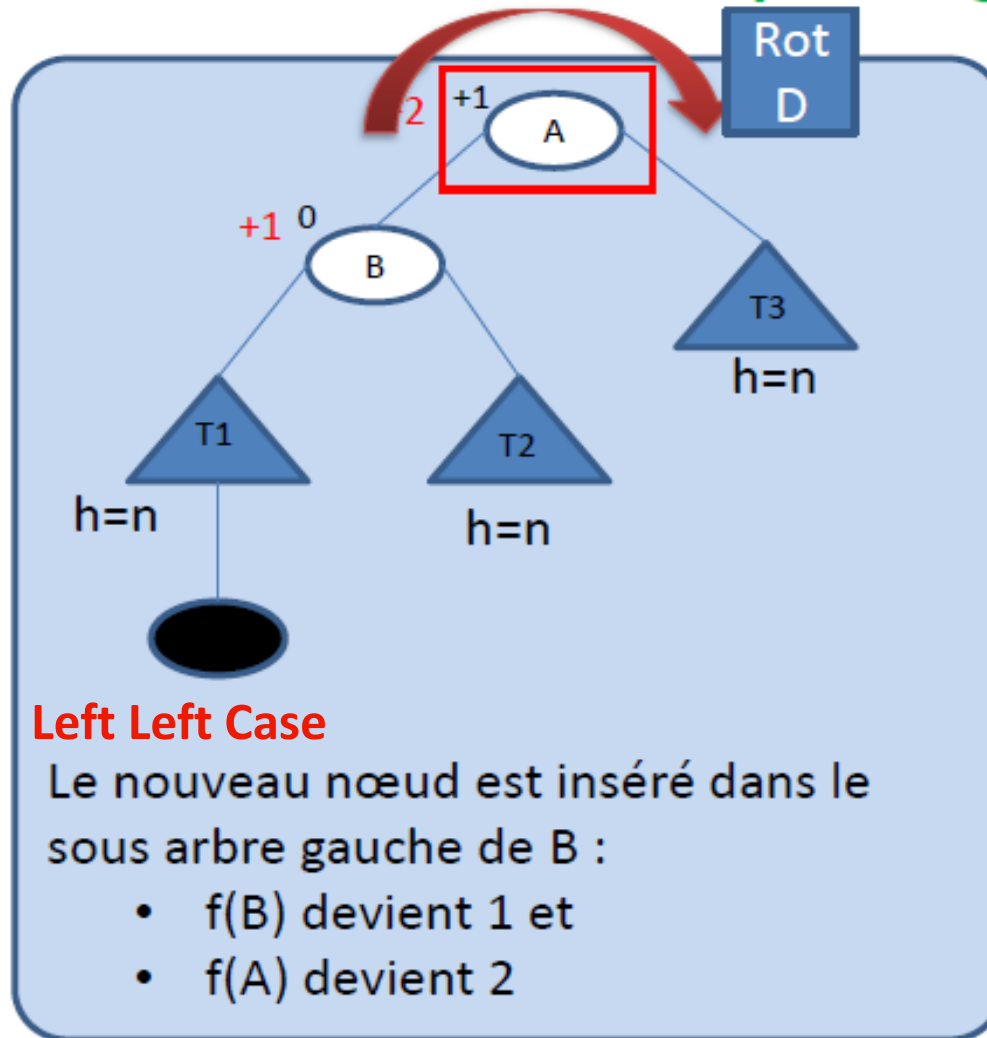
```
// double Right Rotation Case
if (...) {
    node->left = leftRotate(node->left);
    return rightRotate(node);
}
// Double Left Rotation Case
if (...) {
    node->right = rightRotate(node->right);
    return leftRotate(node);
}
```


Arbres binaires de recherche équilibrés

5. Equilibrage suite à l'insertion dans un arbre AVL

Examinons un sous arbre de racine le plus jeune antécédent qui devient non équilibré suite à une insertion

■ Cas où le facteur d'équilibrage est +1



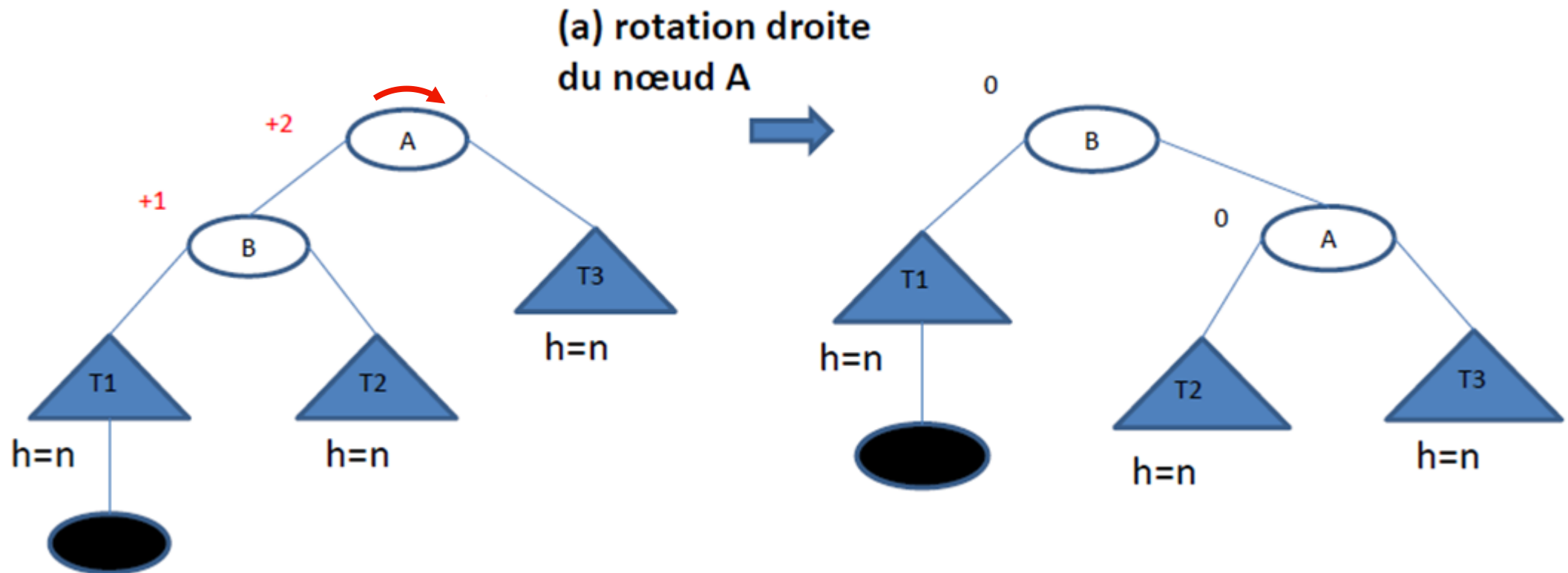
Arbres binaires de recherche équilibrés

Solution

Transformer l'arbre de telle sorte que :

- l'ordre infixe soit préservé
- l'arbre transformé soit équilibré

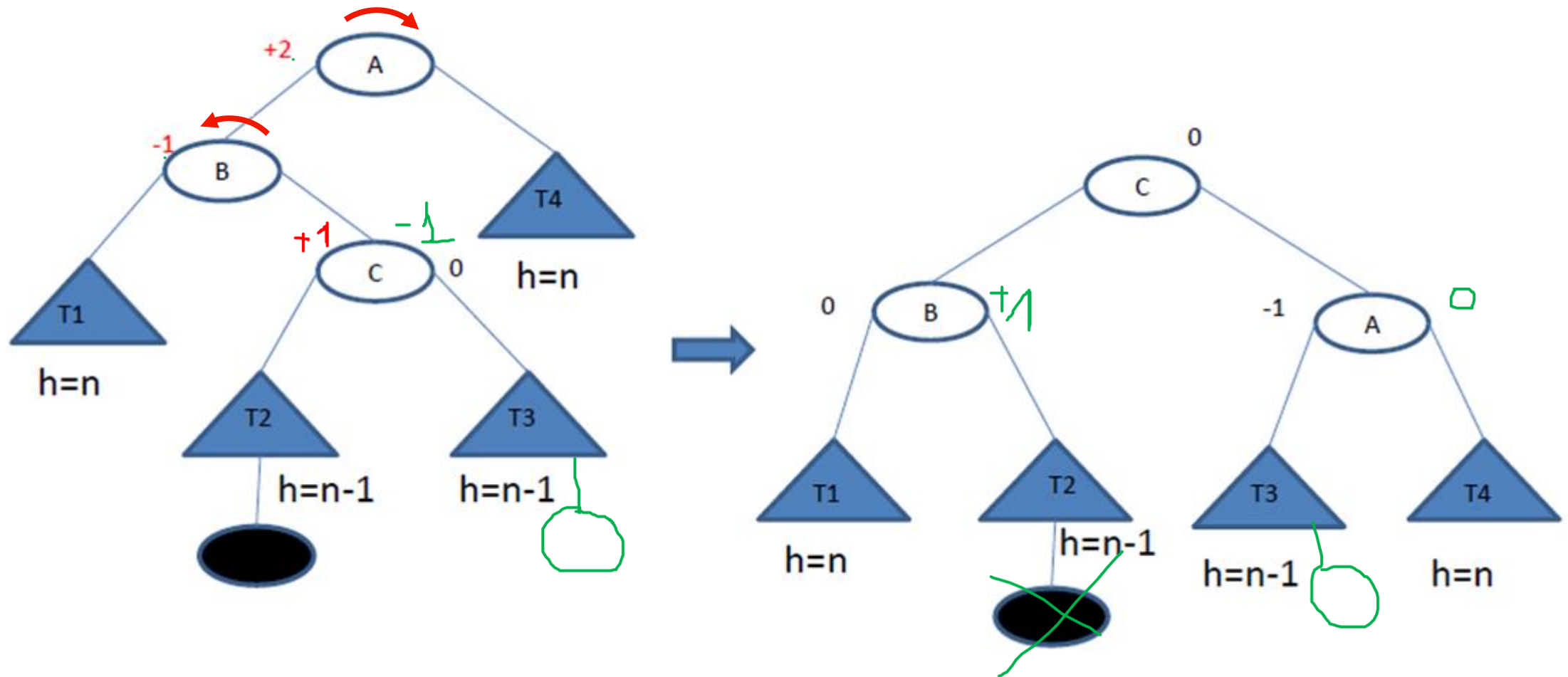
Cas1 (Left Left Case) : $H(SAG(A)) - H(SAD(A)) = 2$ et $H(SAD(B)) \leq H(SAG(B))$



Arbres binaires de recherche équilibrés

Cas 2 (Right Left Case) : $H(SAG(A)) - H(SAD(A)) = 2$ et $H(SAD(B)) > H(SAG(B))$

(b) rotation gauche du nœud B suivie
par une rotation droite du nœud A

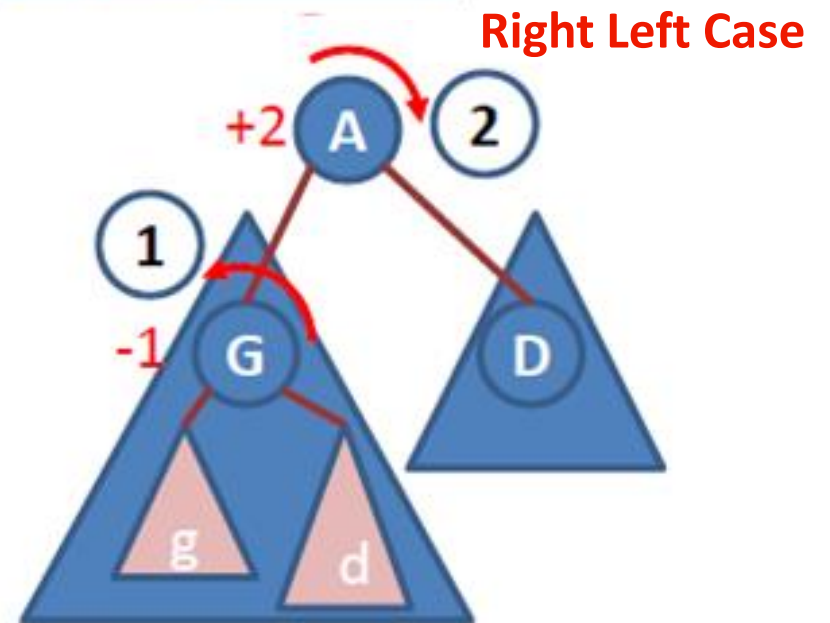
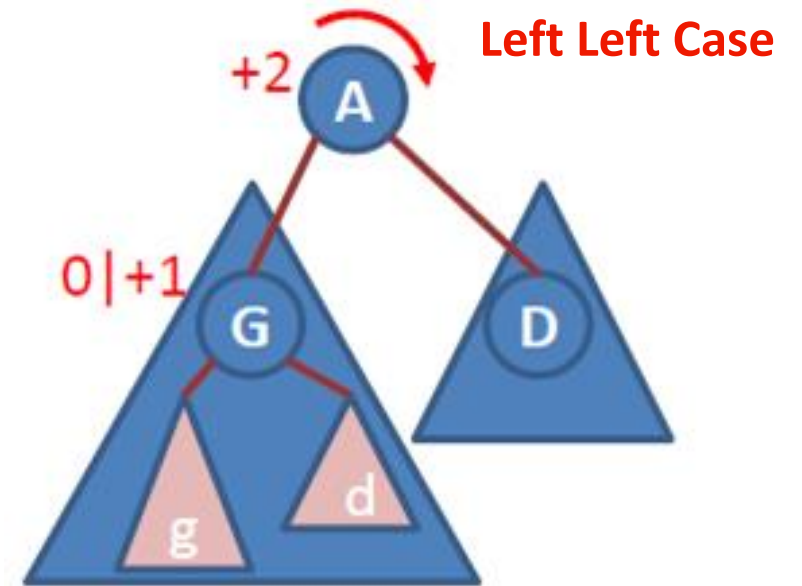


Arbres binaires de recherche équilibrés

Algorithme d'équilibrage

Soit A un arbre dont la racine est le plus jeune antécédent qui devient non équilibré suite à une insertion, G et D ses sous arbres gauche et droit. On suppose que : $H(G) - h(D) = 2$

- Si $h(G) - h(D) = 2$, on fait une rotation droite de A. Soient g et d les sous arbres gauche et droit de G.
 - Si $h(g) < h(d)$, on fait d'abord une rotation gauche de G.
- Si $h(G) - h(D) = -2$, opérations symétriques (en échangeant G par D et g par d)

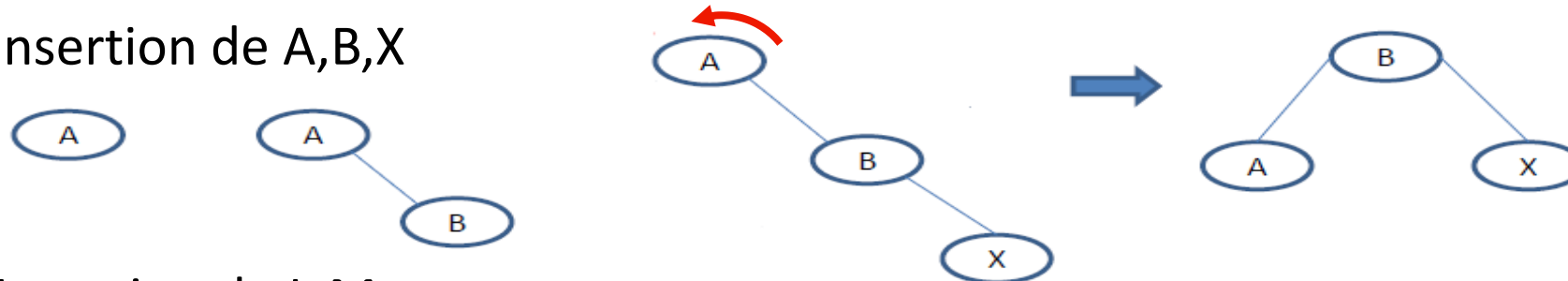


Arbres binaires de recherche équilibrés

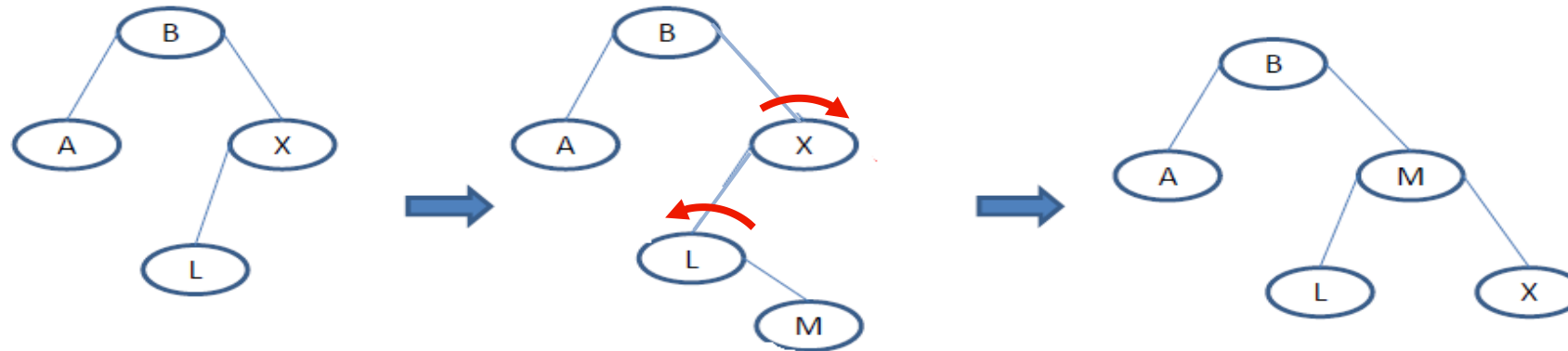
Exemple

Insertion de la séquence : A, B, X, L, M, C, D, E, H, R, F dans un arbre AVL.

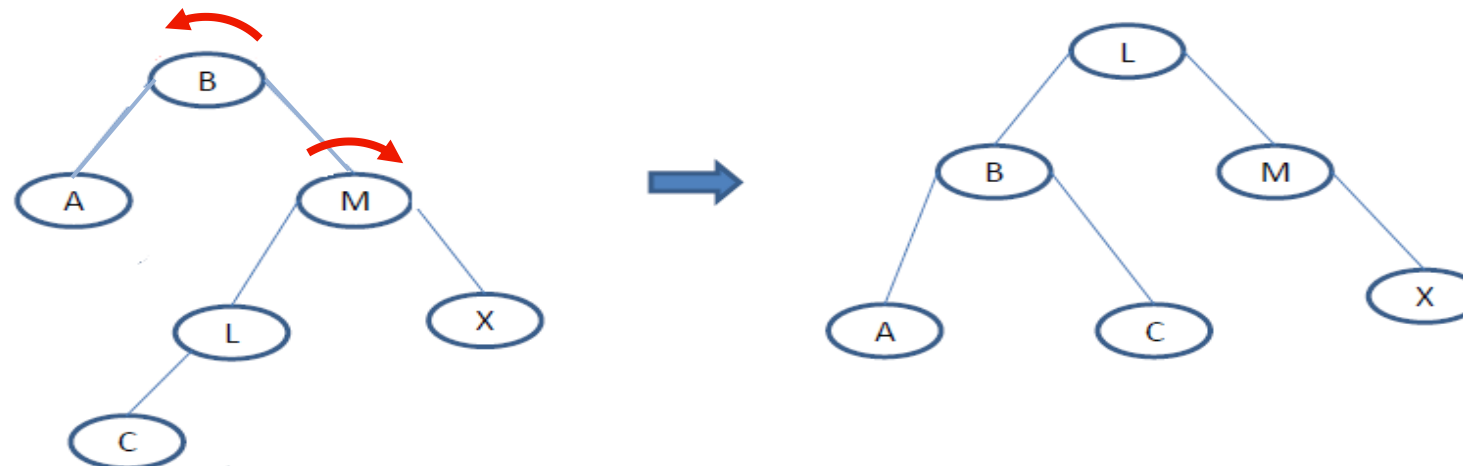
=> Insertion de A,B,X



=> Insertion de L,M

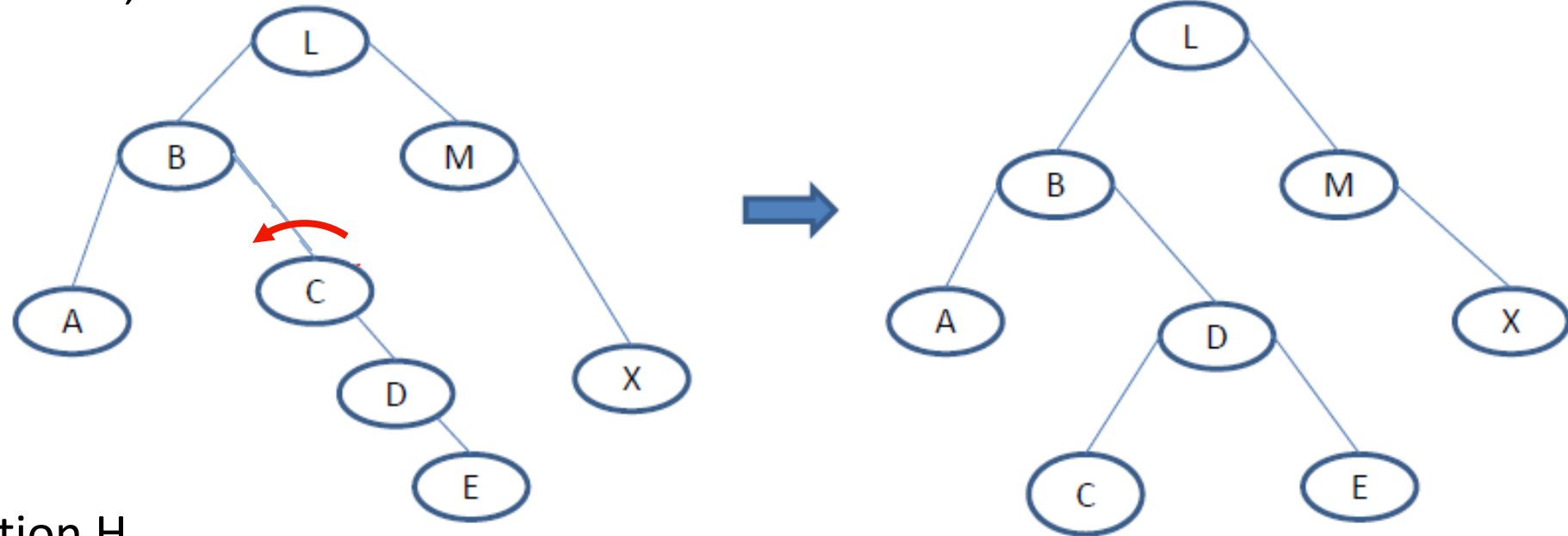


=> Insertion C

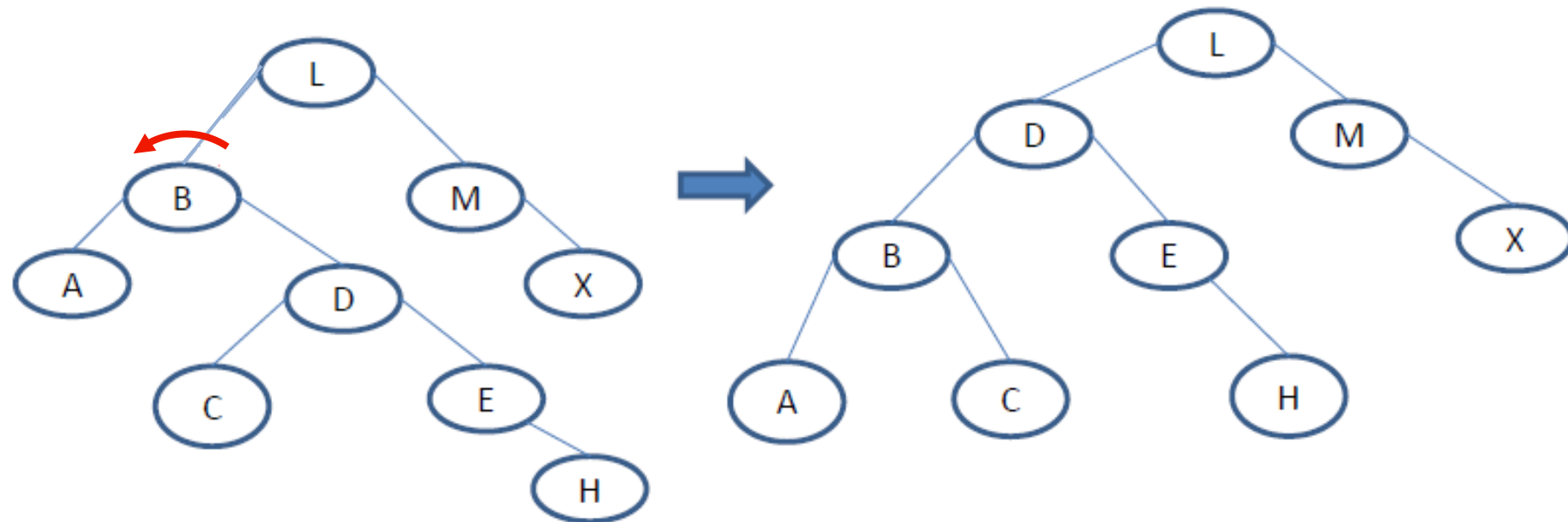


Arbres binaires de recherche équilibrés

=> Insertion D, E

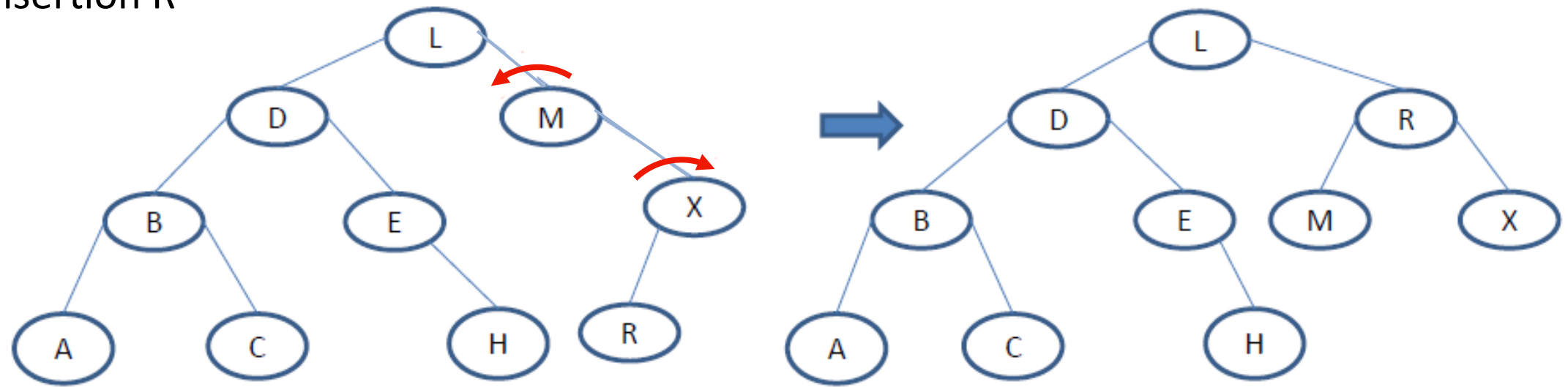


=> Insertion H

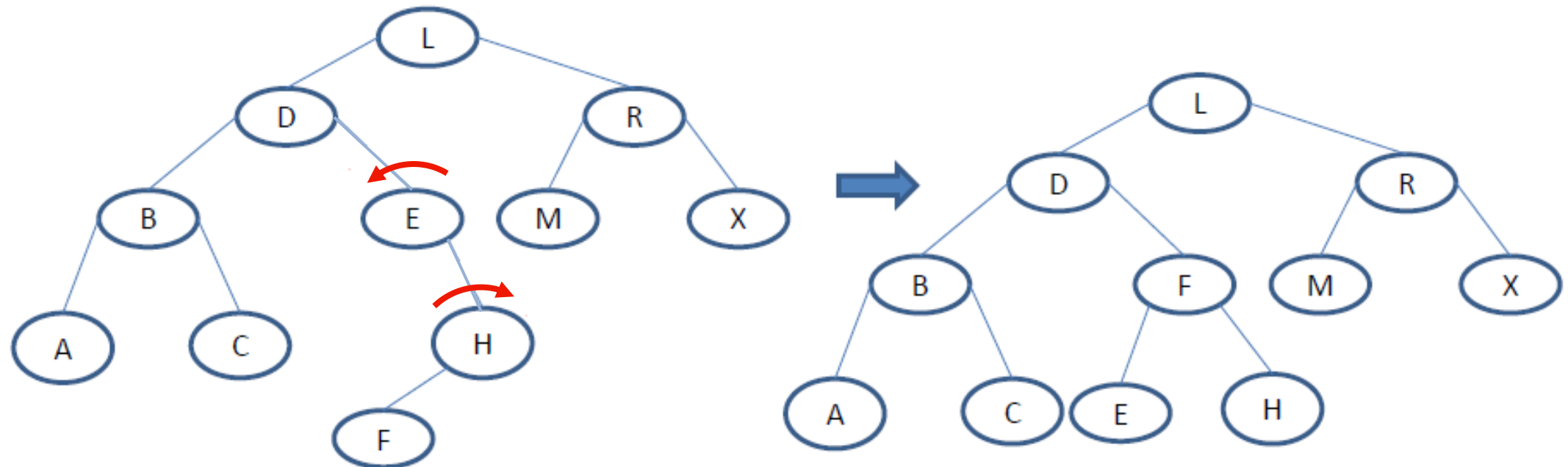


Arbres binaires de recherche équilibrés

=> Insertion R



=> Insertion F



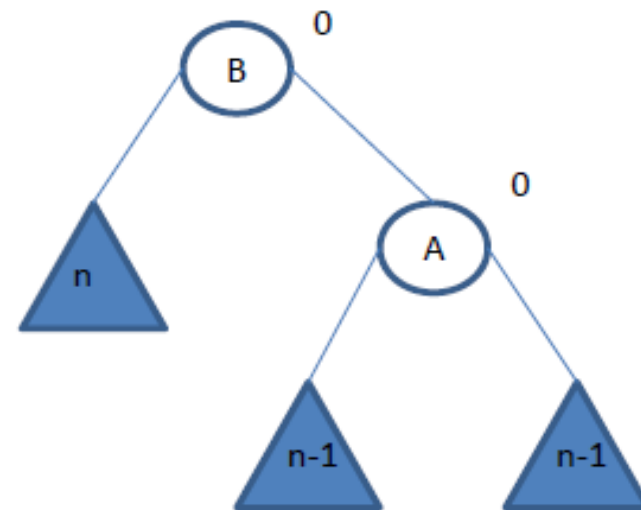
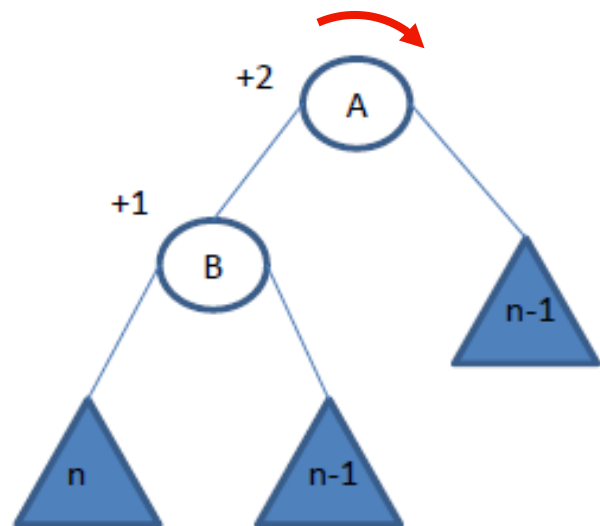
6. Algorithme de Suppression dans un arbre AVL

Principe

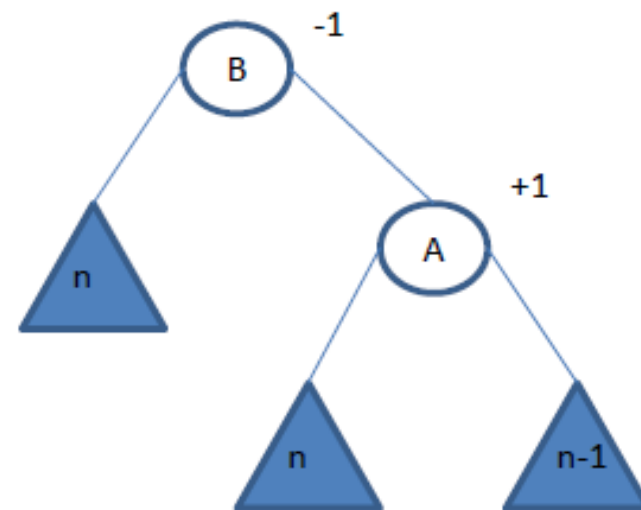
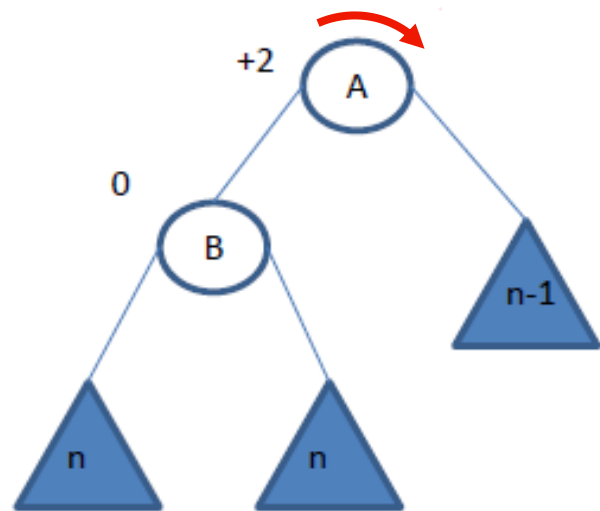
- Étape 1 : supprimer comme dans un arbre de recherche binaire ordinaire
- Étape 2 : mettre à jour les balances
- Étape 3 : Rééquilibrer l'arbre :
 - Cas où la balance d'un nœud A devient $+2 \Rightarrow$ Le fils gauche B de A doit exister
 $\Rightarrow$ Les cas suivants peuvent se présenter
 - a) B a une balance égale à $+1$
 - b) B a une balance égale à 0
 - c) B a une balance égale à -1
 - Traitement symétrique dans le cas où la balance d'un nœud A devient égale -2
 - Traitement peut continuer en cascade du bas en haut

Arbres binaires de recherche équilibrés

Cas a : B a une balance égale à +1

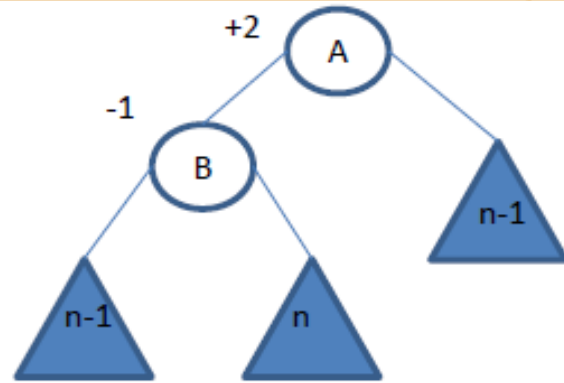


Cas b : B a une balance égale à 0



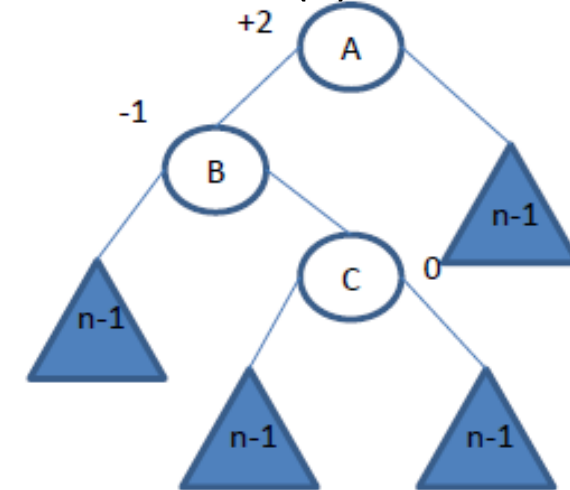
Arbres binaires de recherche équilibrés

Cas c : B a une balance égale à -1

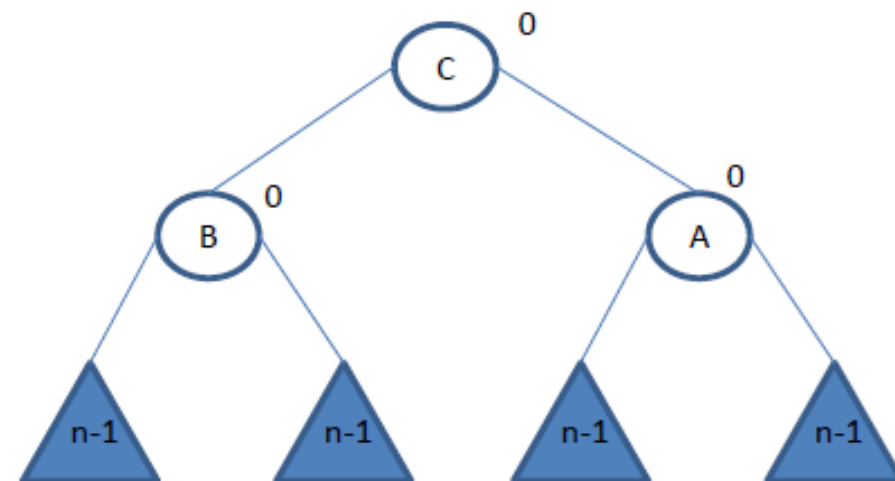
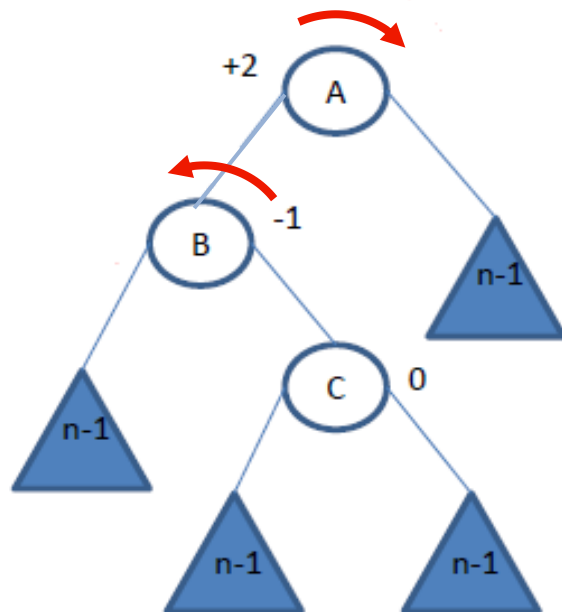


B a donc un fils à sa droite, soit C.

Cas Balance (C) = 0

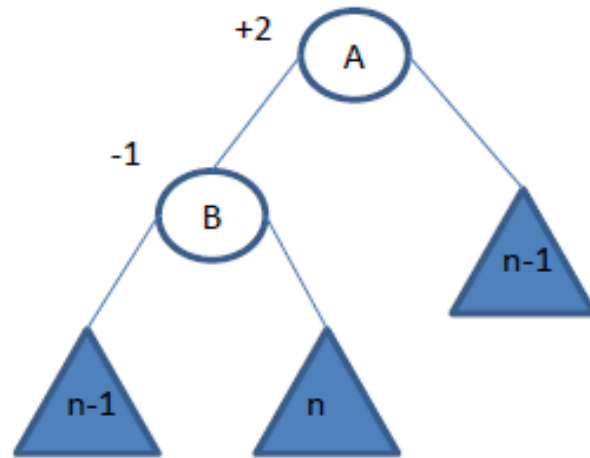


B a une balance égale à -1, C son fils droit avec Balance(C)=0

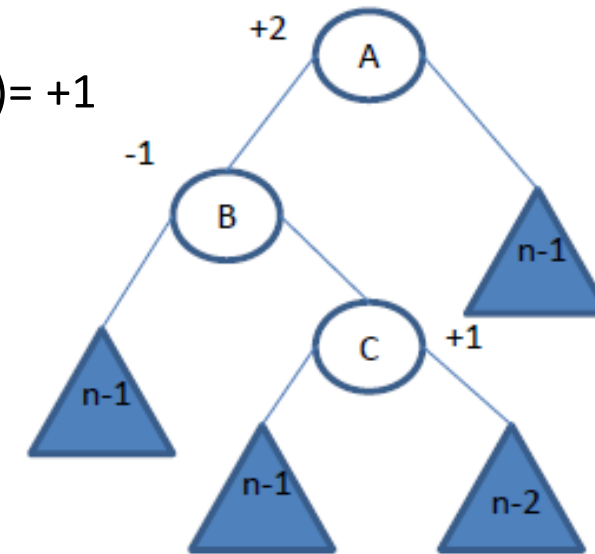


Arbres binaires de recherche équilibrés

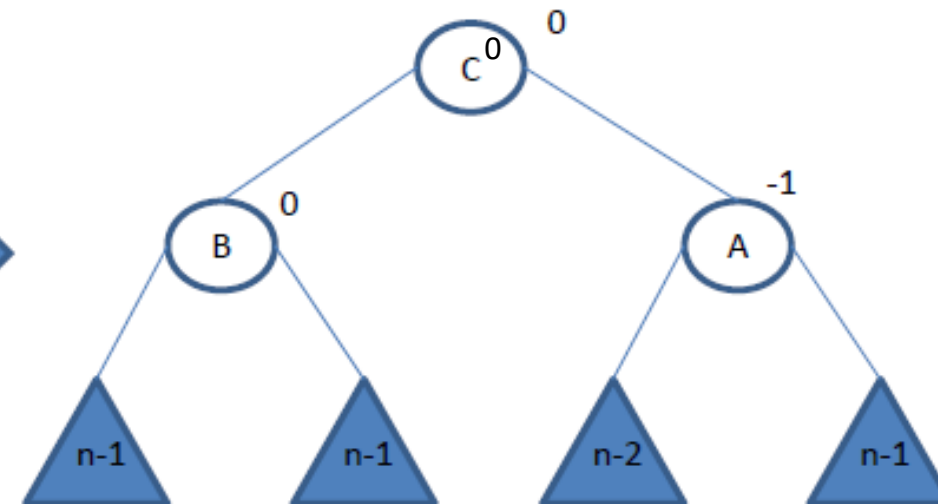
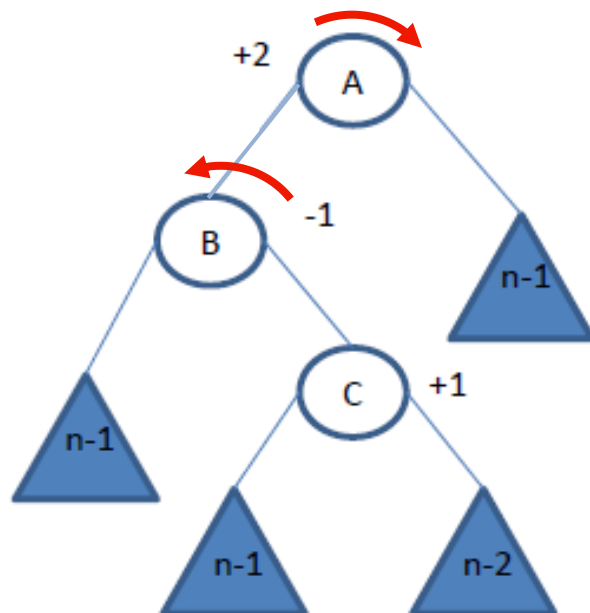
Cas c : B a une balance égale à -1



Balance (C) = +1

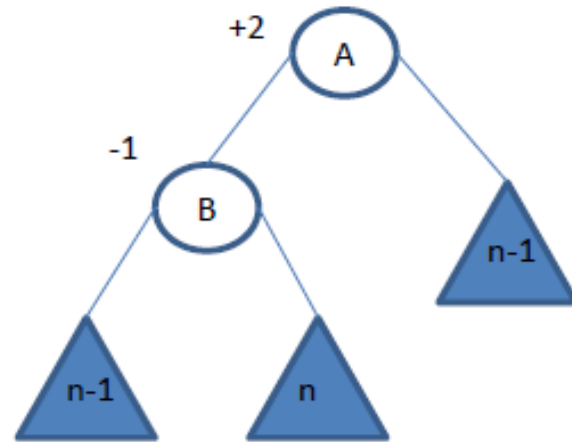


B a une balance égale à -1, C son fils droit avec Balance(C)=+1



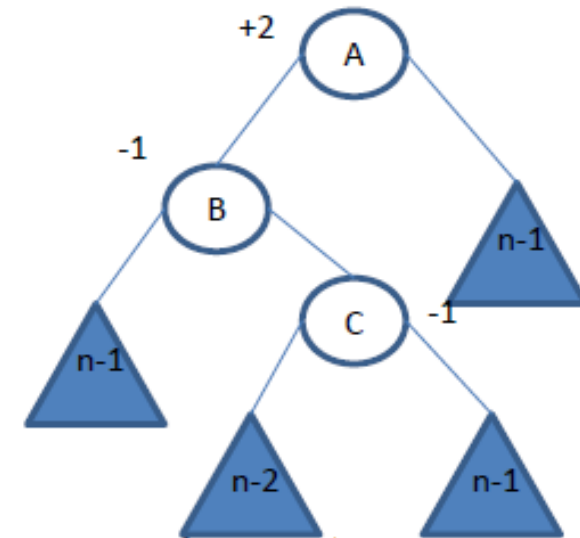
Arbres binaires de recherche équilibrés

Cas c : B a une balance égale à -1

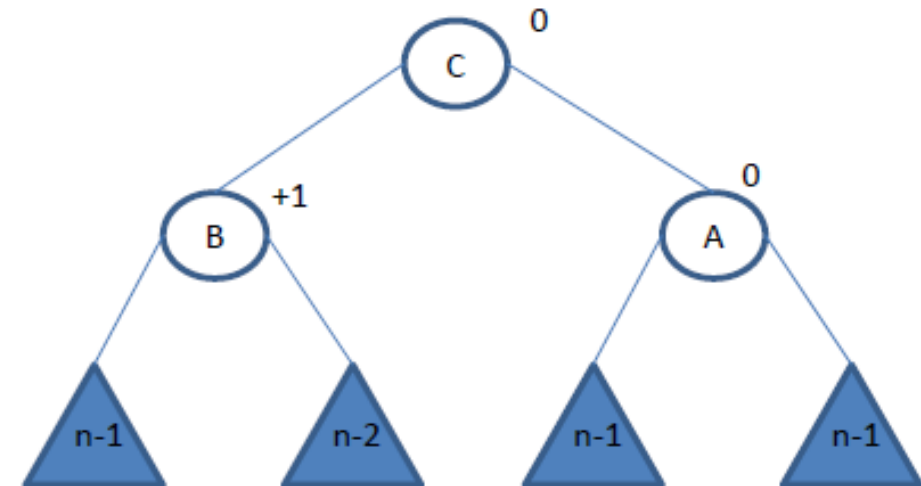
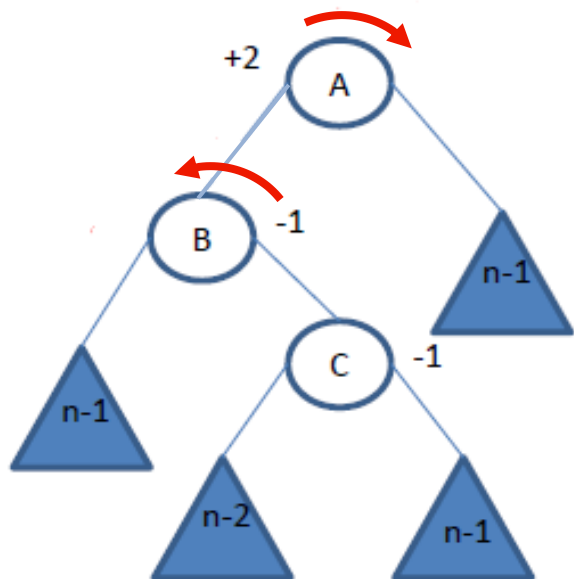


Balance (C) = -1

=



B a une balance égale à -1, C son fils droit avec Balance(C)=-1



Arbres binaires de recherche équilibrés

6. Implémentation C/C++ d'un arbre binaire de recherche équilibré d'entiers

Planter un Arbre Binaire permettant de stocker une collection d'entiers en prévoyant les opérations :

- Création
- Calcul de la Hauteur de l'arbre
- Insertion
- Suppression
- Recherche
- Affichage de l'arbre binaire en infixe, préfixé, postfixé

Arbres binaires de recherche équilibrés

■ Insertion dans un arbre AVL

Code

```
Node* insert(Node* node, int data) {
    // comme pour l'insertion dans un ABR
    if (node == NULL) return newNode(data);
    if (data < node->data)
        node->left = insert(node->left, data);
    else if (data > node->data) node->right = insert(node->right, data);
    else return node; // cas sans doublons
    node->height = 1 + max(height(node->left), height(node->right));
    int balance = getBalance(node);
    // Left Left Case
    if (balance > 1 && data < node->left->data) return rightRotate(node);
    if (balance > 1 && data > node->left->data) { // Left Right Case
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
    // Right Right Case
    if (balance < -1 && data > node->right->data) return leftRotate(node);
    if (balance < -1 && data < node->right->data) { // Right Left Case
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }
    return node;
}
```

Avec:

```
int getBalance(Node* node) {
    if (node == NULL) return 0;
    return height(node->left)
        - height(node->right);
}

int height(Node* node)
{
    if (node == NULL) return -1;
    return node->height;
}
```

Arbres binaires de recherche équilibrés

■ Suppression dans un arbre AVL

Code

```
Node* deleteNode(Node* root, int data) {
    // Comme pour la suppression dans un ABR
    if (root == NULL) return root;
    if (data < root->data) root->left = deleteNode(root->left, data);
    else if (data > root->data) root->right = deleteNode(root->right, data);
    else {
        if ((root->left == NULL) || (root->right == NULL)) { // Node with only one child or no child
            Node* temp = root->left ? root->left : root->right;
            if (temp == NULL) { // No child case
                free(root);
                root = NULL;
            }
            else { // One child case
                *root = *temp;
                free(temp);
            }
        }
        else { // Node with two children: Get the inorder successor
            Node* temp = minValueNode(root->right);
            root->data = temp->data; // Copy the inorder successor's data to this node
            root->right = deleteNode(root->right, temp->data); // Delete the inorder successor
        }
    }
}
```

Arbres binaires de recherche équilibrés

■ Suppression dans un arbre AVL

Code(suite)

```
if (root == NULL) return root; // If the tree had only one node then return
root->height = 1 + max(height(root->left), height(root->right)); // Update height of the current node
int balance = getBalance(root); // Get balance factor of the current node
```

Avec:

// Les 4 cas de déséquilibre

// Left Left Case

if (balance > 1 && getBalance(root->left) >= 0) return rightRotate(root);

if (balance > 1 && getBalance(root->left) < 0) { // Left Right Case

root->left = leftRotate(root->left);

return rightRotate(root);

}

// Right Right Case

if (balance < -1 && getBalance(root->right) <= 0) return leftRotate(root);

if (balance < -1 && getBalance(root->right) > 0) { // Right Left Case

root->right = rightRotate(root->right);

return leftRotate(root);

}

return root;

}

Node* minValueNode(Node* node)

{

Node* current = node;

while (current->left != NULL)

current = current->left;

return current;

}

Avec:

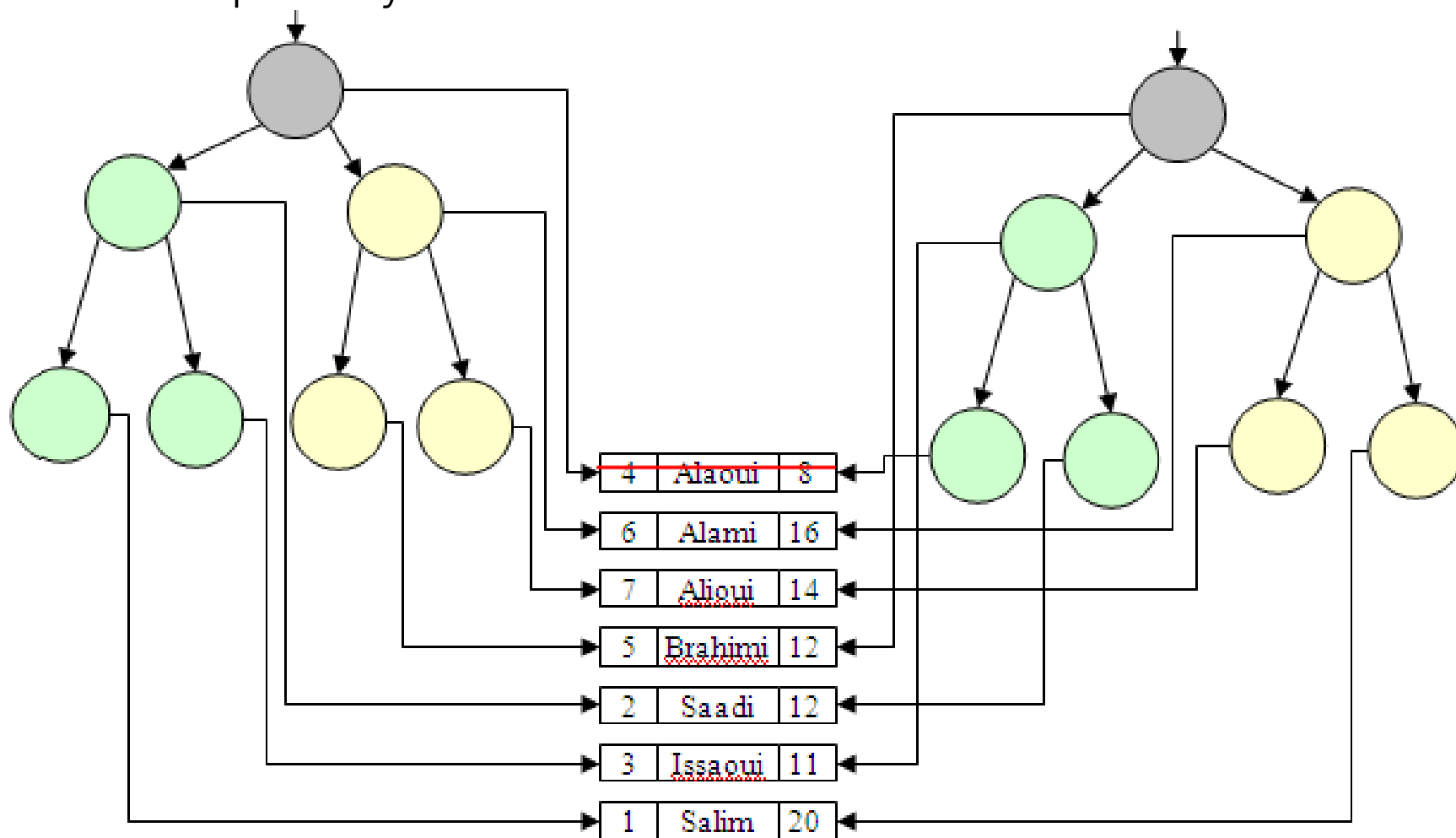
Complexité des différentes opérations :

Opération	Complexité
Recherche dans un arbre AVL	$O(h)=O(\log n)$
Insertion dans un arbre AVL	$O(h)=O(\log n)$
suppression dans un arbre AVL	$O(h)=O(\log n)$

Applications

Application 1 - recherche dans une liste selon deux clés

Réaliser une application qui implémente la structure de données suivante pour permettre des recherches sur une liste dynamique d'étudiants suivant deux clés : recherche par matricule et recherche par moyenne.



Applications

Application 2

1. Enoncé

On souhaite gérer une liste d'étudiants connu chacun par :

- Un matricule entier numéroté automatiquement
- Un nom
- Une liste de notes
- Une moyenne calculable.

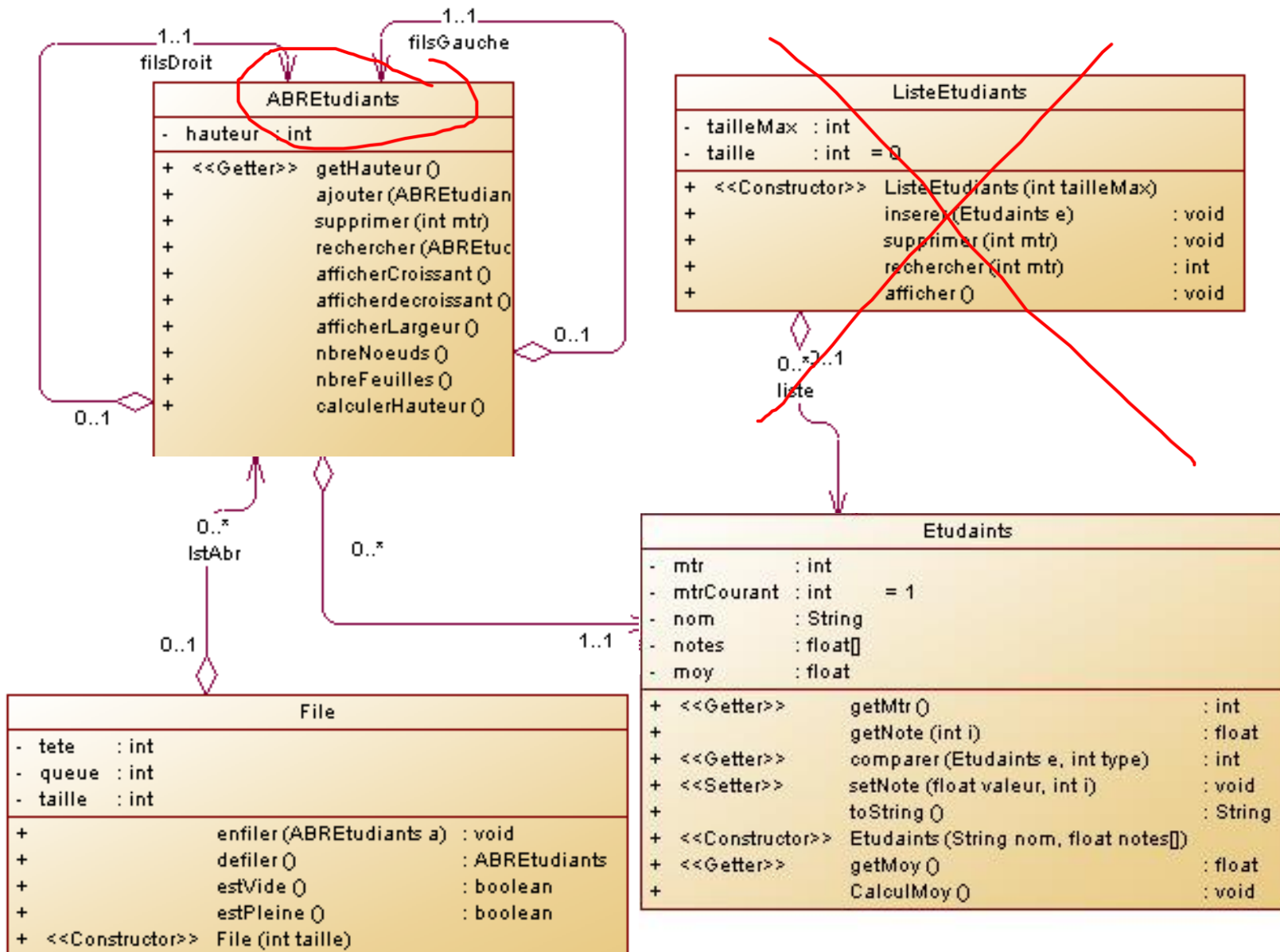
Cette gestion doit permettre de manipuler la liste par les opérations d'insertion, de recherche, de suppression , de tri,....

Cette gestion est prévue par les deux organisations suivantes :

- **Solution 1 :** Cas d'une liste d'étudiants représentée par un ABR AVL ordonné par matricule
- *Solution 2 : Cas d'une liste d'étudiants représentée par un tableau ordonné par matricule*

Applications

Le diagramme de classe suivant donne l'ensemble des classes nécessaires pour la gestion souhaitée :



Applications

2. Travail à faire

- 2.1. Donner l'implémentation C++ de la classe Etudiants
- 2.2. Donner l'implémentation de la classe ListeEtudiants
- 2.3. Donner l'implémentation de la classe File
- 2.4. Donner l'implémentation de la classe ABREtudiants
- 2.5. Ecrire un jeu de test pour tester l'ensemble des classes